

MICROCONTRÔLEUR (SIMULATION SOUS PROTEUS ET PRATIQUE)



I. INTRODUCTION

Le développement technologique a permis la miniaturisation des composants électroniques facilitant ainsi leur exploitation et leur implémentation dans les montages électroniques. Cela a eu pour conséquence la mise en place de systèmes électroniques de taille réduite.

L'un des principaux composants au cœur des systèmes électroniques de nos jours est le microcontrôleur. En abrégé **μc** , c'est un ensemble de circuits intégrés constituant les éléments essentiels d'un ordinateur. Ces éléments sont un processeur, des mémoires, des unités périphériques et des interfaces entrées-sorties.

Ce cours est une initiation à la mise en place de montages électroniques développés autour d'un microcontrôleur. En vue de mieux comprendre le fonctionnement d'un microcontrôleur, nous le programmerons et le simulerons en utilisant le logiciel de simulation dénommé « PROTEUS ». Son originalité réside dans sa capacité à simuler en temps réel les circuits électroniques et également à concevoir les typons. Notez que la mise en place de typons ne sera pas abordée dans cette formation.

OBJECTIFS

Au terme de cette formation, les participants devront être capables de réaliser et simuler des systèmes à base de microcontrôleur : le microcontrôleur adopté sera programmé et simulé en utilisant le logiciel « **PROTEUS** ». Cela permettra aux participants de contempler le fonctionnement en temps réel d'un montage réalisé sur « **PROTEUS** ».

II. EXEMPLES DE MICROCONTRÔLEURS

- Les PICS de Microchip : **PIC16F84; 16F876; 16F877; etc.**
- Les MCS de Intel : **MCS8051; MCS8052; D2000; etc.**
- Les μc de Motorola : **MC68HC11; MC68HC12; MC68HC711D3; etc.**
- Les μc Arduino : **Arduino UNO; Arduino Mega; Arduino Leonardo; etc.**

III. PRÉSENTATION DE ARDUINO

ARDUINO est une plateforme combinant matériels et logiciels pour le développement d'applications embarquées. La carte électronique est constituée de composants électroniques autour d'un microprocesseur (**ATMEL AVR**). Elle comporte un ensemble d'entrées et de sorties selon les versions de la carte.

Quant au logiciel utilisé pour la programmation de la carte, c'est un mélange de **C** et **C++** adapté aux possibilités de la carte.



Figure 1 : Carte ARDUINO



Figure 2 : environnement de développement

III. PRÉSENTATION DE ARDUINO

1. La carte ARDUINO

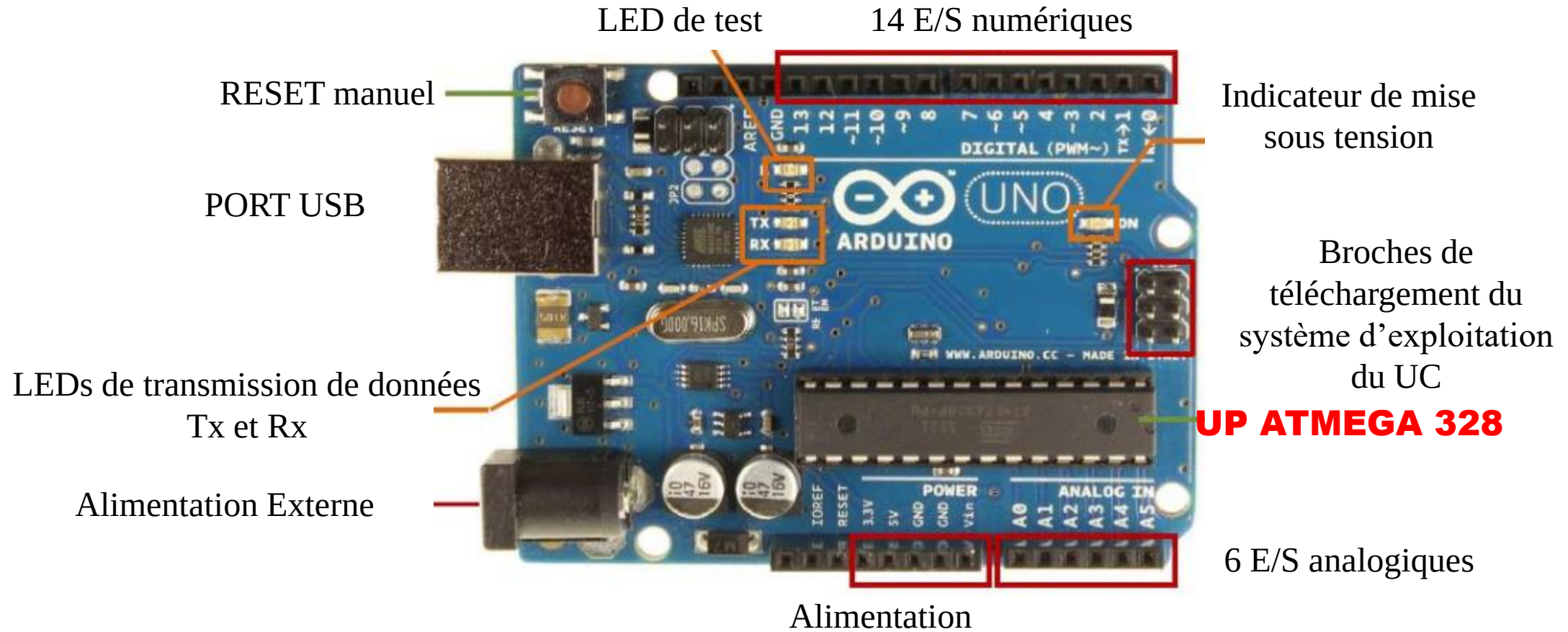


Figure 3 : Carte ARDUINO

III. PRÉSENTATION DE ARDUINO

1. La carte ARDUINO

Tableau 1 : les principales caractéristiques de la carte Arduino UNO

Microprocesseur	ATMEGA 328
Tension d'alimentation interne	5V
Plage de l'alimentation recommandée	7 à 12V
Plage de l'alimentation limite	6 à 20V
E/S numériques	14 dont 6 PWM configurables
Entrées Analogiques	6
Courant Max par broche	40 mA (200 mA pour E/S cumulées)
Fréquence horloge	16 MHz
Mémoire Flash	32 Ko dont 0,5 Ko utilisées par le Bootloader
Mémoire SRAM (volatile)	2KB
Mémoire EEPROM (non volatile)	1KB

N.B : **PWM** = **P**ulse **W**idth **M**odulation. Les entrées/sorties analogiques peuvent être utilisées comme des entrées/sorties numériques

III. PRÉSENTATION DE ARDUINO

1. La carte ARDUINO

Tableau 2 : les caractéristiques des E/S numériques

Numéro de la broche	Port	Fonction selon le paramétrage
0	PD0	Rx : entrée liaison série
1	PD1	Tx : sortie liaison série
2	PD2	INT0 : entrée interruption externe
3	PD3	INT1 : entrée interruption externe
4	PD4	T0: Entrée Timer/compteur 0 et XCK entrée horloge
5	PD5	T1 : Entrée Timer/compteur 1 et OC0B sortie PWM
6	PD6	OC0A sortie PWM et AIN0 Entrée comparateur analogique
7	PD7	AIN1 Entrée comparateur analogique
8	PB0	CLKO sortie de l'horloge et ICP1 entrée de capture Timer/Compteur 1
9	PB1	OC1A : sortie PWM
10	PB2	SS slave liaison SPI et OC1B : sortie PWM

III. PRÉSENTATION DE ARDUINO

1. La carte ARDUINO

Tableau 3 : les caractéristiques des E/S numériques

Numéro de la broche	Port	Fonction selon le paramétrage
11	PB3	MOSI sortie liaison SPI et OC2A sortie PWM
12	PB4	MISO entrée liaison SPI
13	PB5	SCK : horloge liaison SPI

N.B : les broches numériques 0 (Rx) et 1 (Tx) sont utilisées pour une communication série. La carte supporte par ailleurs une liaison série synchrone grâce aux broches 10 (SS), 11 (MOSI), 12 (MISO) et 13 (SCX).

III. PRÉSENTATION DE ARDUINO

1. La carte ARDUINO

La carte possède 6 entrées/sorties analogiques repérées **A0** à **A5** (PC0 à PC5) pouvant admettre une tension analogique maximale de 5V configurable. Les entrées A4 et A5 peuvent être utilisées respectivement comme la ligne de donnée SDA et la ligne d'horloge de l'interface série I2C. Ces broches peuvent également être utilisées comme des broches numériques.

N.B : l'utilisation des broches analogiques en broches numériques est réalisée via une programmation à travers l'environnement de développement.

III. PRÉSENTATION DE ARDUINO

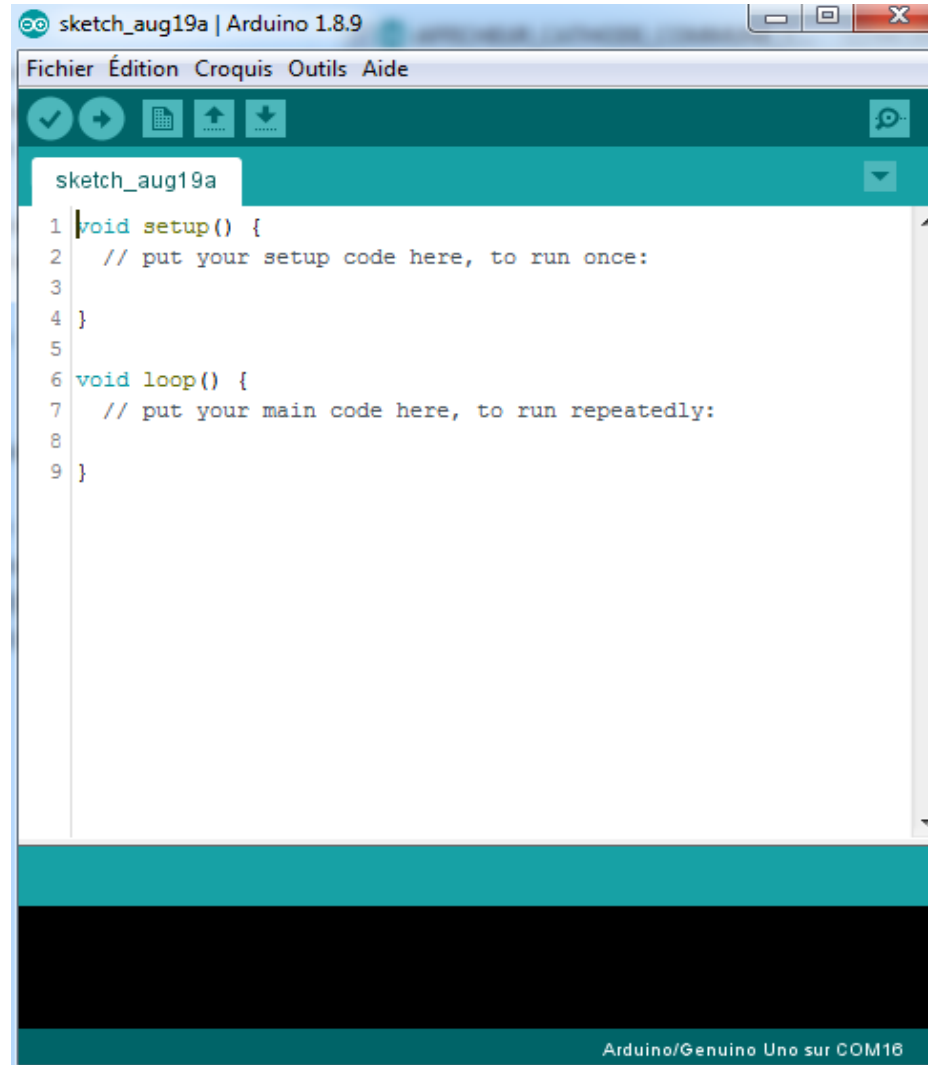
2. Le logiciel de programmation

Libre et multiplateformes, le logiciel ARDUINO permet d'écrire et compiler des programmes . Il permet en outre de se connecter et de communiquer avec la carte ARDUINO pour y télécharger les programmes préalablement compilés (sans retour d'erreurs).

Il peut être téléchargé sur le site **www.arduino.cc**

III. PRÉSENTATION DE ARDUINO

2. Le logiciel de programmation



III. PRÉSENTATION DE ARDUINO

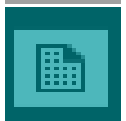
2. Le logiciel de programmation



- Pour compiler le programme afin de vérifier les erreurs de syntaxe.



- Pour télécharger le programme dans la carte ARDUINO.



- Pour ouvrir une nouvelle fenêtre.



- Pour sauvegarder un programme.



- Pour ouvrir un ancien programme.

III. PRÉSENTATION DE ARDUINO

2. Le logiciel de programmation

Le logiciel de programmation ARDUINO est composé de trois grandes parties :

- la première débutant avant « **void setup ()** » permet de définir les variables et constantes; elle permet également d'inclure des bibliothèques;
- la deuxième qui est « **void setup ()** » contient les instructions d'initialisation. Cette partie ne s'exécute qu'une seule fois : par exemple, la configuration des entrées/sorties;
- la troisième partie définie dans « **void loop ()** » contient les instructions qui seront exécutées indéfiniment jusqu'à ce que la carte ARDUINO soit mise hors tension.

IV. PROGRAMMATION DE ARDUINO

1. Définition des variables

Une variable est définie selon le type, lequel reposant sur les valeurs minimale et maximale pouvant être sauvegardées par cette dernière. Nous avons :

Appellation	Description
boolean	Donnée logique (vrai ou faux) sur 8 bits
byte	Entier non signé sur 8 bits
int	Entier signé sur 16 bits
unsigned int	Entier non signé sur 16 bits
long	Entier signé sur 32 bits
unsigned long	Entier non signé sur 32 bits
float	Nombre à virgule flottant sur 32 bits
char	Caractère sur 8 bits

Exemple : « **byte var** » définit var comme une variable entière non signée sur 8 bits.

IV. PROGRAMMATION DE ARDUINO

2. Définition des constantes

Les constantes sont définies en utilisant les fonction **#define** ou **const** .

Exemple : #define var 100 ou const byte var = 100

Des constantes permettant de rapidement configurer les broches du microprocesseur ATMEGA ont été définies :

Désignation	Description
true	Donnée logique égale à 1
false	Donnée logique égale à 0
HIGH	Génération d'un niveau de tension haut sur les E/S numériques
LOW	Génération d'un niveau de tension bas sur les E/S numériques
INPUT	Configuration d'une broche numérique en entrée
OUTPUT	Configuration d'une broche numérique en sortie

IV. PROGRAMMATION DE ARDUINO

3. Les opérateurs

a. Les opérateurs arithmétiques

Appellation	Symboles	Exemple
Addition	+	$x = a + b;$
Soustraction	-	$x = a - b;$
Multiplication	*	$x = a * b;$
Division	/	$X = a / b;$
Modulo	%	$X = a \% b;$

N.B : modulo est le reste d'une division entière : $x = 2 \% 4 = 2.$

IV. PROGRAMMATION DE ARDUINO

3. Les opérateurs

b. Les affectations

Appellation	Symboles	Exemple
Affectation ordinaire	=	a = b;
Additionner de	+=	a += b ou a = a + b;
Soustraire de	-=	a -= b ou a = a - b;
Multiplier par	*=	a *= b ou a = a * b;
Diviser par	/=	a /= b ou a = a / b;
Modulo par	%=	a %= b ou a = a % b;
ET avec	&=	a &= b ou a = a & b;
OU avec	=	a = b ou a = a b;
Décrémentation de 1	--	a -- ou a = a - 1;
Incrémentation de 1	++	a ++ ou a = a + 1;

IV. PROGRAMMATION DE ARDUINO

3. Les opérateurs

c. Les opérateurs de comparaison

Appellation	Symboles	Exemple
Egal à	==	a == b;
Diffèrent de	!=	a != b ;
ET logique	&&	a &&b;
OU logique		a b;
Supérieur à	>	a > b;
Inférieur à	<	a < b;
Supérieur ou égal à	>=	a >= b;
Inférieur ou égal à	<=	a <= b;

IV. PROGRAMMATION DE ARDUINO

4. Les fonctions

a. La gestion des Entrées/Sorties numériques

- **pinMode (pin,Mode)** permet de configurer la broche numérique de numéro « **pin** » en entrée ou en sortie. « **Mode** » peut prendre 3 valeurs : **INPUT**, **OUTPUT** ou **INPUT_PULLUP**.

Exemple : pinMode (5,OUTPUT) définit la broche numérique PD5 en sortie

IV. PROGRAMMATION DE ARDUINO

4. Les fonctions

a. La gestion des Entrées/Sorties numériques

- **digitalWrite (pin,Value)** génère un niveau de tension défini par «**Value** » sur la broche de numéro « **pin** ». « **Value** » peut être **HIGH** ou **LOW**.

Exemple : digitalWrite (5,HIGH) écrit un niveau de tension haut sur PD5.

N.B : avant toute utilisation de digitalWrite, il faudrait que le mode de la broche ait été défini via la fonction pinMode.

IV. PROGRAMMATION DE ARDUINO

4. Les fonctions

a. La gestion des Entrées/Sorties numériques

- **digitalRead (pin)** retourne le niveau de tension d'une broche. La valeur lue est soit **HIGH** soit **LOW**.

Exemple : digitalRead (4) récupère le niveau de tension de la broche 4.

N.B : avant toute utilisation de digitalRead, il faudrait que le mode de la broche ait été défini via la fonction pinMode.

IV. PROGRAMMATION DE ARDUINO

4. Les fonctions

b. La gestion des Entrées/Sorties analogiques

- **analogWrite (pin,Value)** écrit le niveau de tension défini par « **Value** » sur la broche analogique de numéro « **pin** ».

Exemple : analogWrite (5,200) écrit la valeur 200 sur la broche A5.

N.B : **Value** varie de 0 à 1023 et est de type **int**. Avant utilisation de analogWrite, pas besoin de définir auparavant le mode de la broche en utilisant pinMode. Le signal en sortie est de type PWM.

IV. PROGRAMMATION DE ARDUINO

4. Les fonctions

b. La gestion des Entrées/Sorties analogiques

- **analogRead (pin)** lit le niveau de tension présent sur la broche analogique de numéro « **pin** ».

Exemple : analogRead (5).

N.B : la valeur retournée est de type **int** et est comprise entre 0 et 1023.

IV. PROGRAMMATION DE ARDUINO

4. Les fonctions

b. La gestion des Entrées/Sorties analogiques

N.B : il est possible de configurer les Entrées/Sorties analogiques en broches numériques. Dans ce cas, leur définition avec **pinMode** est précédée de **A**.

Exemple : `pinMode (A0,OUTPUT);`
`digitalWrite (A0,LOW);`

IV. PROGRAMMATION DE ARDUINO

4. Les fonctions

c. La gestion du temps

- **delay (ms)** : permet de faire une pause de ms en millisecondes. La donnée à rentrer doit être de type **unsigned long**.
- **delayMicroseconds (us)** : permet de faire une pause de us en microsecondes. La donnée à rentrer doit être de type **unsigned int**.
- **micro ()** : retourne le temps en microsecondes à partir du moment où la carte ARDUINO a été mise en marche. La donnée retournée est de type « **unsigned long** »
- **millis ()** : retourne le temps en millisecondes à partir du moment où la carte ARDUINO a été mise en marche. La donnée retournée est de type « **unsigned long** »

IV. PROGRAMMATION DE ARDUINO

4. Les fonctions

d. La gestion des interruptions externes temps

- **attachInterrupt (digitalPinToInterrupt(pin),ISR,Mode)** : permet d'activer une interruption sur la broche de numéro « **pin** ». Lorsque cette interruption est activée, la fonction exécutée est « **ISR** » selon le mode « **Mode** ». Mode peut être : LOW (active l'interruption lorsque la broche reçoit un niveau de tension faible), CHANGE lorsque la broche change de valeur, RISING lorsqu'un front montant est enregistré et FALLING lorsqu'un front descendant est enregistré.

N.B : la variable utilisée dans la fonction **ISR** doit être de type **volatile**.

IV. PROGRAMMATION DE ARDUINO

4. Les fonctions

d. La gestion des interruptions externes

- **detachInterrupt (digitalPinToInterrupt(pin))** : permet de désactiver une interruption activée sur une broche de numéro « pin ».

IV. PROGRAMMATION DE ARDUINO

4. Les fonctions

e. Les structures

- **La condition « si » :**

```
if (condition)
{
}
else if (condition)
{
}
else
{
}
```

Exemple :

```
if (i <= 25)
{
    digitalWrite (5,HIGH);
}
else
{
    digitalWrite (5,LOW);
}
```

IV. PROGRAMMATION DE ARDUINO

4. Les fonctions

e. Les structures

- **La boucle « do ... while » :**

```
do
{
}
while (condition);
```

Exemple :

```
do
{
    i ++;
}
while (i<25);
```

IV. PROGRAMMATION DE ARDUINO

4. Les fonctions

e. Les structures

- **La boucle « while » :**

```
while (condition)
{
}
```

Exemple :

```
while (i<25)
{
    i ++;
}
```

IV. PROGRAMMATION DE ARDUINO

4. Les fonctions

e. Les structures

- **La boucle « for » :**

```
for (byte i = 0; i < 4; i++)  
{  
}
```

Exemple :

```
for (byte i = 0; i < 7; i++)  
{  
    digitalWrite (i, HIGH);  
}
```


IV. PROGRAMMATION DE ARDUINO

4. Les fonctions

e. Les structures

- **La condition « switch » :**

```
switch (var)
{
case 1:
    break;

case 2:
    break;

    default:
    break;
}
```

Exemple :

```
switch (i)
{
    case 1 : digitalWrite (3,HIGH);
    break;
    default : digitalWrite (3,LOW);
    break;
}
```

IV. PROGRAMMATION DE ARDUINO

4. Les fonctions

e. Les structures

- **L'instruction « goto » :**

Label :

goto Label;

permet de se connecter à une étiquette.

IV. PROGRAMMATION DE ARDUINO

4. Les fonctions

e. Les structures

- **L'instruction « return »** : Permet d'arrêter l'exécution normale d'une fonction et de retourner une valeur si elle est spécifiée.

Exemples : `return;`
 ou
 `return 2;`

IV. PROGRAMMATION DE ARDUINO

4. Les fonctions

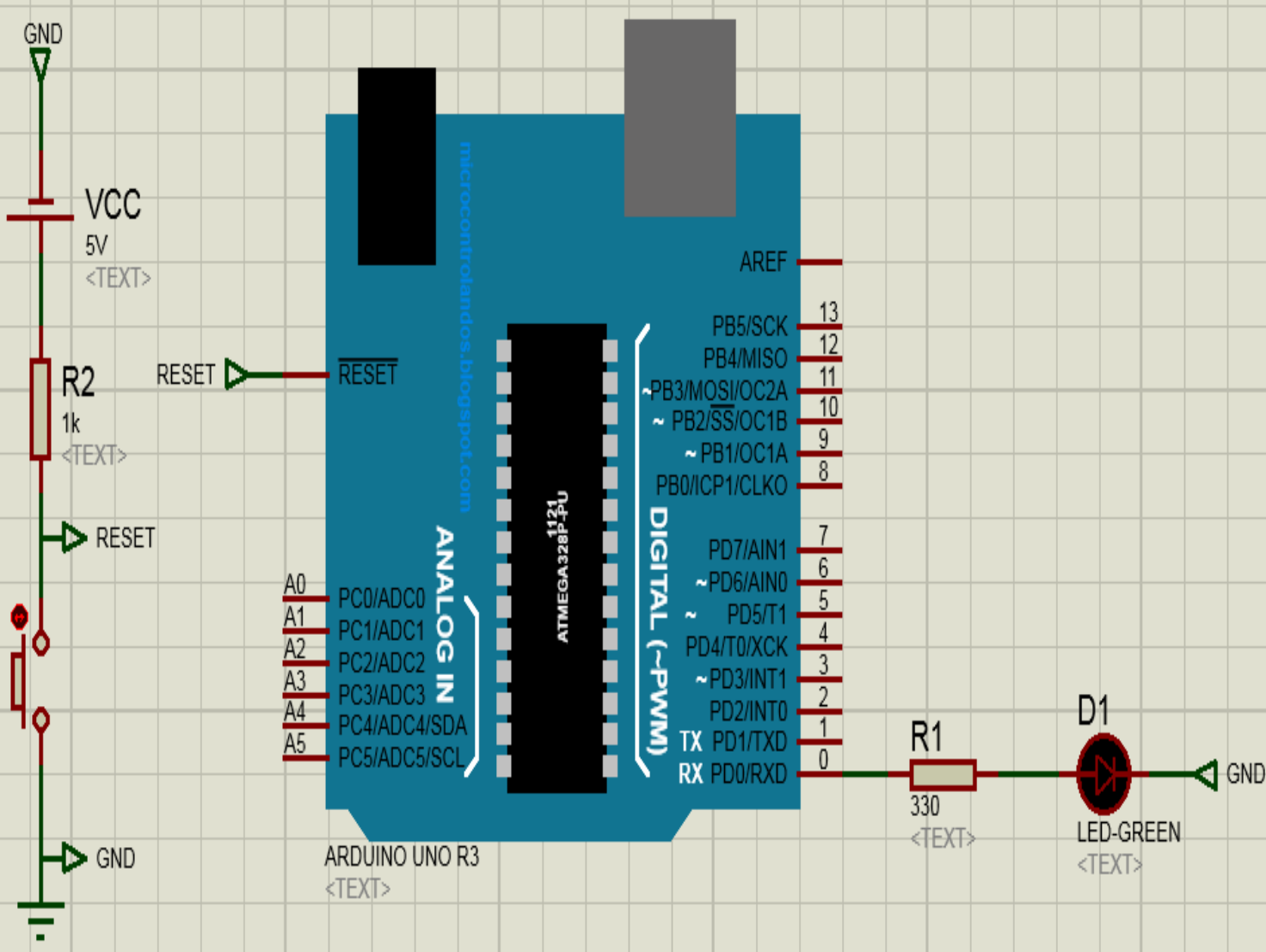
f. Insertion de commentaires

- `//` : permet de commenter une ligne de codes;
- `/* */` : permet de commenter un bloc de commentaires.

EXERCICE I: clignotement d'une LED

TAF : réalisez et programmez un montage à base d'ARDUINO permettant de faire clignoter une LED placée sur la broche PD0.

EXERCICE I_SOLUTION



```
#define Led 0  
int Temps = 1; // EN SECONDES
```

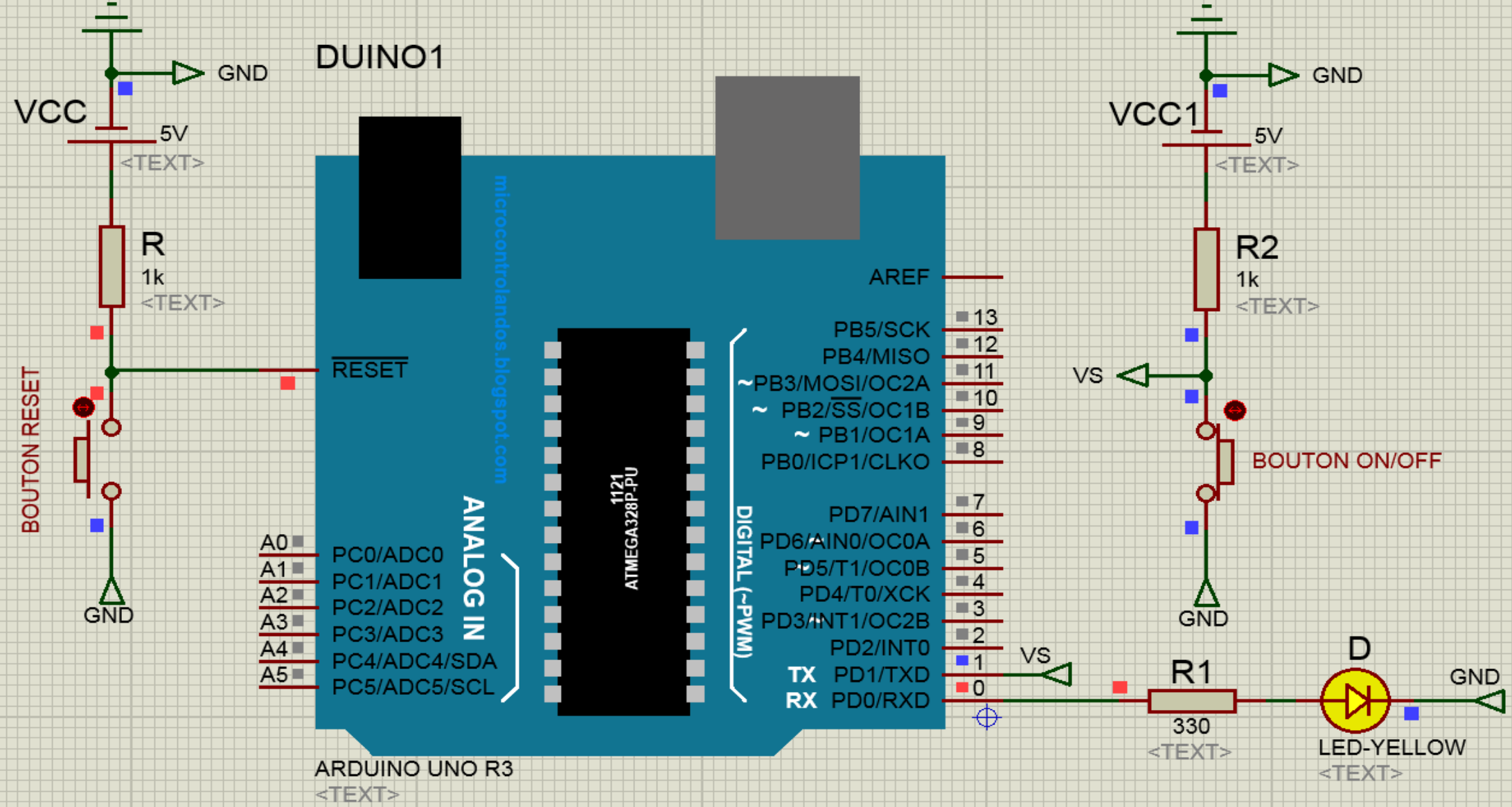
```
void setup()  
{  
  pinMode (Led,OUTPUT);  
}
```

```
void loop()  
{  
  digitalWrite (Led,HIGH);  
  delay (Temps * 1000);  
  
  digitalWrite (Led,LOW);  
  delay (Temps * 1000);  
}
```

EXERCICE II: ALLUMAGE D'UNE LED VIA UN BP

TAF : réalisez et programmez un circuit électronique à base d'ARDUINO permettant d'allumer une LED placée sur la broche PD0 lorsqu'un bouton poussoir placé sur la broche PD1 est au niveau de tension bas sinon extinction de cette dernière.

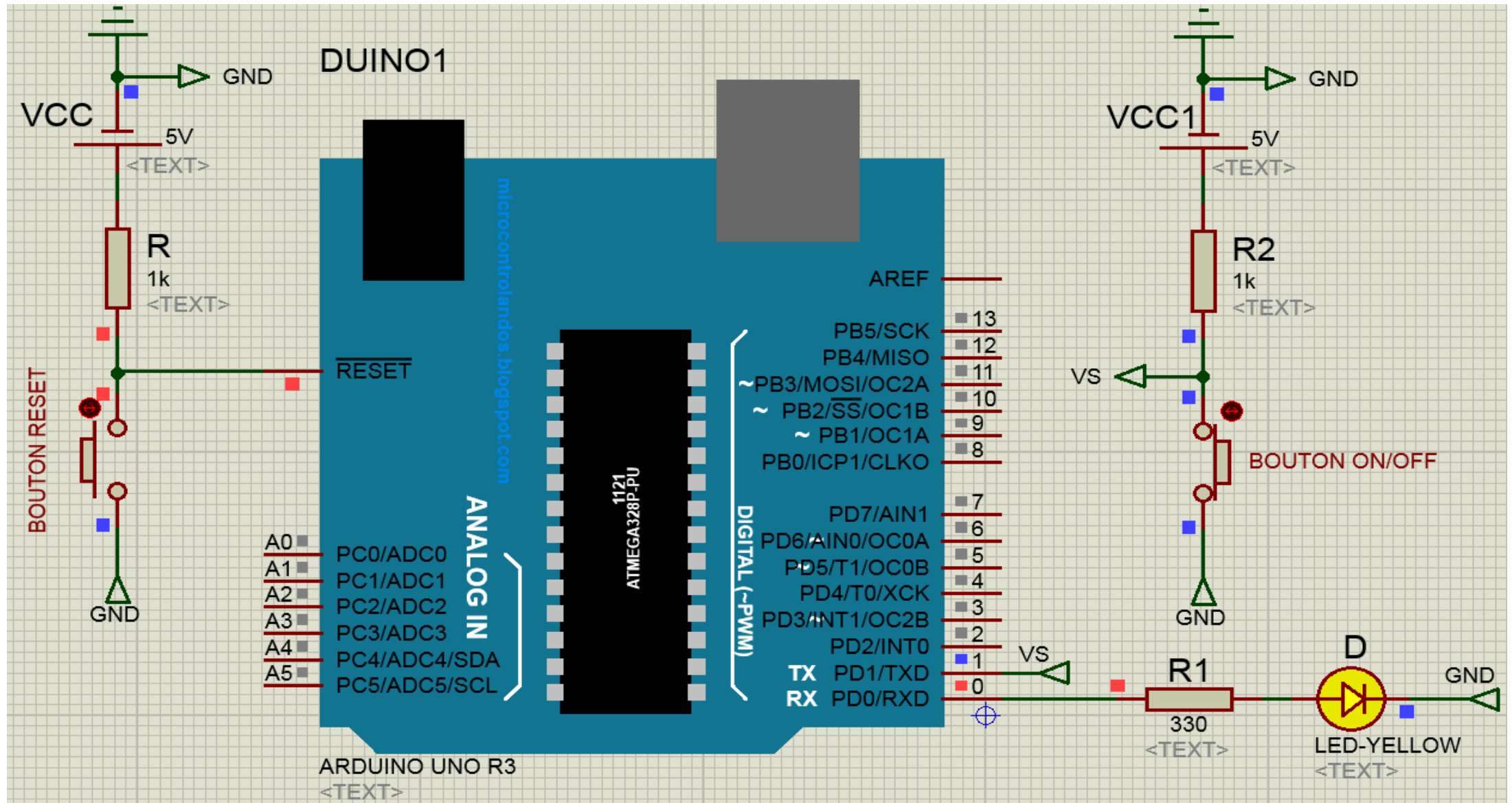
EXERCICE II_SOLUTION



EXERCICE III: CLIGNOTEMENT D'UNE LED

TAF : réalisez et programmez un circuit électronique à base d'ARDUINO permettant de faire clignoter (allumage/extinction) une LED placée sur la broche PD0 lorsque le bouton poussoir placé sur la broche PD1 a un niveau de tension bas; dans le cas contraire le clignotement s'arrête.

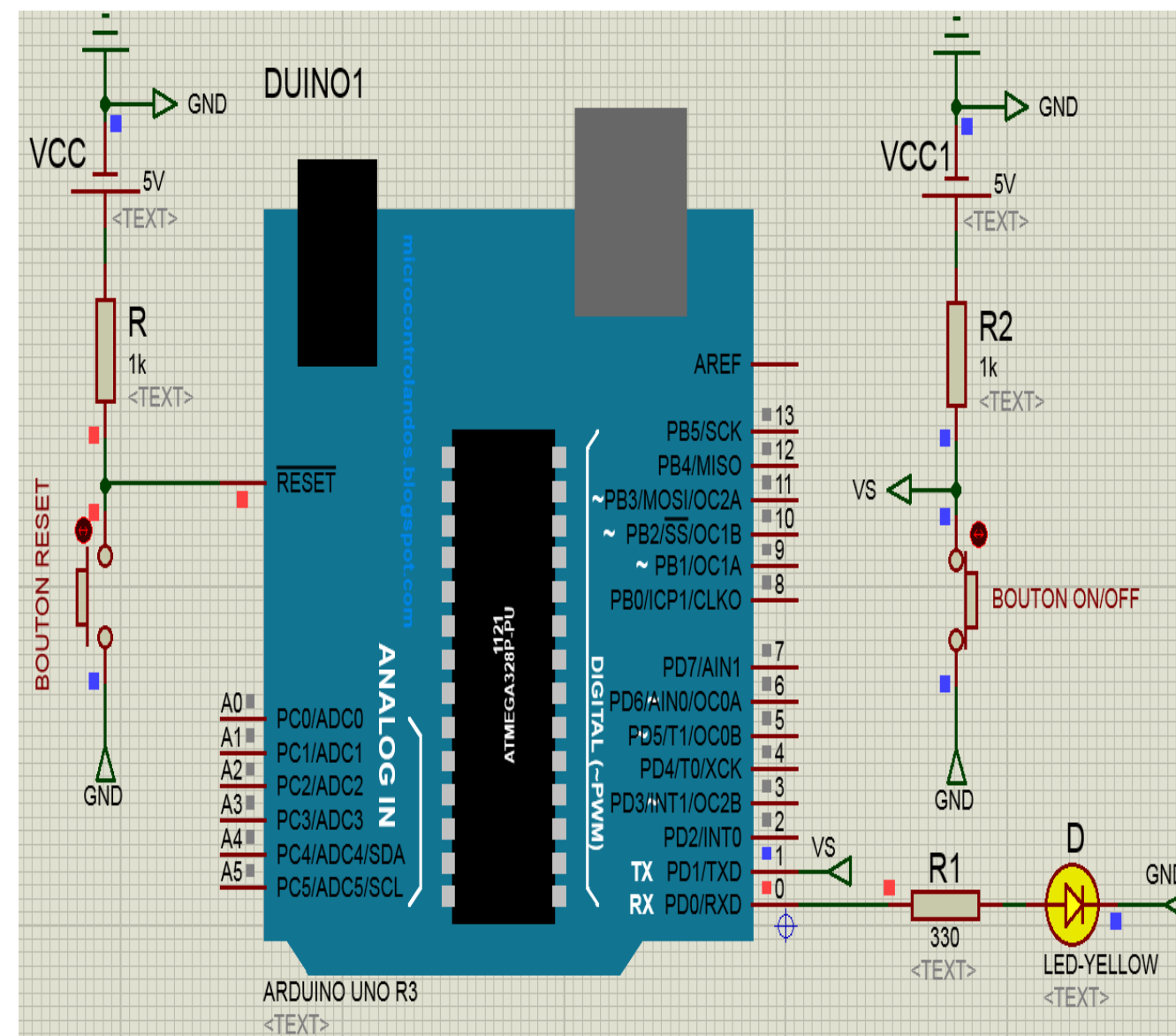
EXERCICE III_SOLUTION



EXERCICE IV: CONTRÔLE D'UNE LED VIA UN BP

TAF : réalisez et programmez un montage à base d'ARDUINO permettant de changer l'état (allumage/extinction) d'une LED placée sur la broche PD0 à chaque appui et relâche d'un bouton poussoir placé sur la broche PD1.

EXERCICE IV_SOLUTION



```
#define Led 0
```

```
#define Bouton_Poussoir 1
```

```
boolean Etat_Led = LOW;
```

```
byte Etat_Appui = 0;
```

```
void setup()
```

```
{
  pinMode (Led, OUTPUT);
  pinMode (Bouton_Poussoir, INPUT);
  digitalWrite (Led, Etat_Led);
}
```

```
void loop()
```

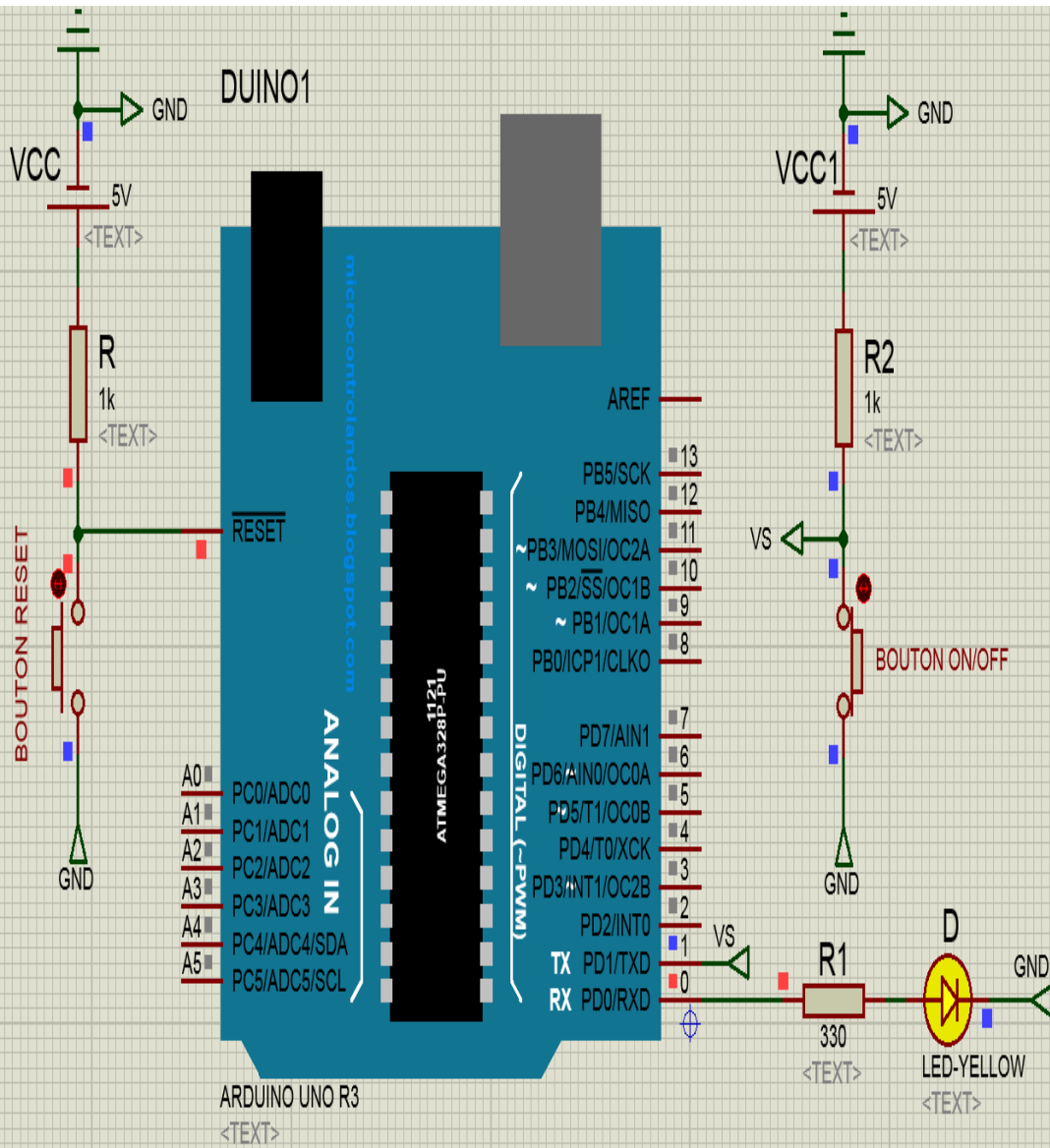
```
{
  if (digitalRead(Bouton_Poussoir) == LOW && Etat_Appui == 0)
  {
    Etat_Appui = 1;
  }

  if (digitalRead(Bouton_Poussoir) == HIGH && Etat_Appui == 1)
  {
    Etat_Led = !Etat_Led;
    digitalWrite(Led, Etat_Led);
    Etat_Appui = 0;
  }
}
```

EXERCICE V: CONTRÔLE D'UNE LED VIA UN BP_IE

TAF : reprenez la tâche de l'exercice IV en utilisant une interruption externe sur PD2.

EXERCICE V_SOLUTION



```
#define Led 0
```

```
#define Bouton_Poussoir 2
```

```
boolean volatile Etat_Led = LOW;
```

```
void setup()
```

```
{
```

```
  pinMode (Led, OUTPUT);
```

```
  pinMode (Bouton_Poussoir, INPUT);
```

```
  digitalWrite (Led, Etat_Led);
```

```
  attachInterrupt (digitalPinToInterrupt (Bouton_Poussoir), Changement_Etat_Led, RISING);  
}
```

```
void loop()
```

```
{
```

```
  digitalWrite(Led, Etat_Led);
```

```
}
```

```
void Changement_Etat_Led()
```

```
{
```

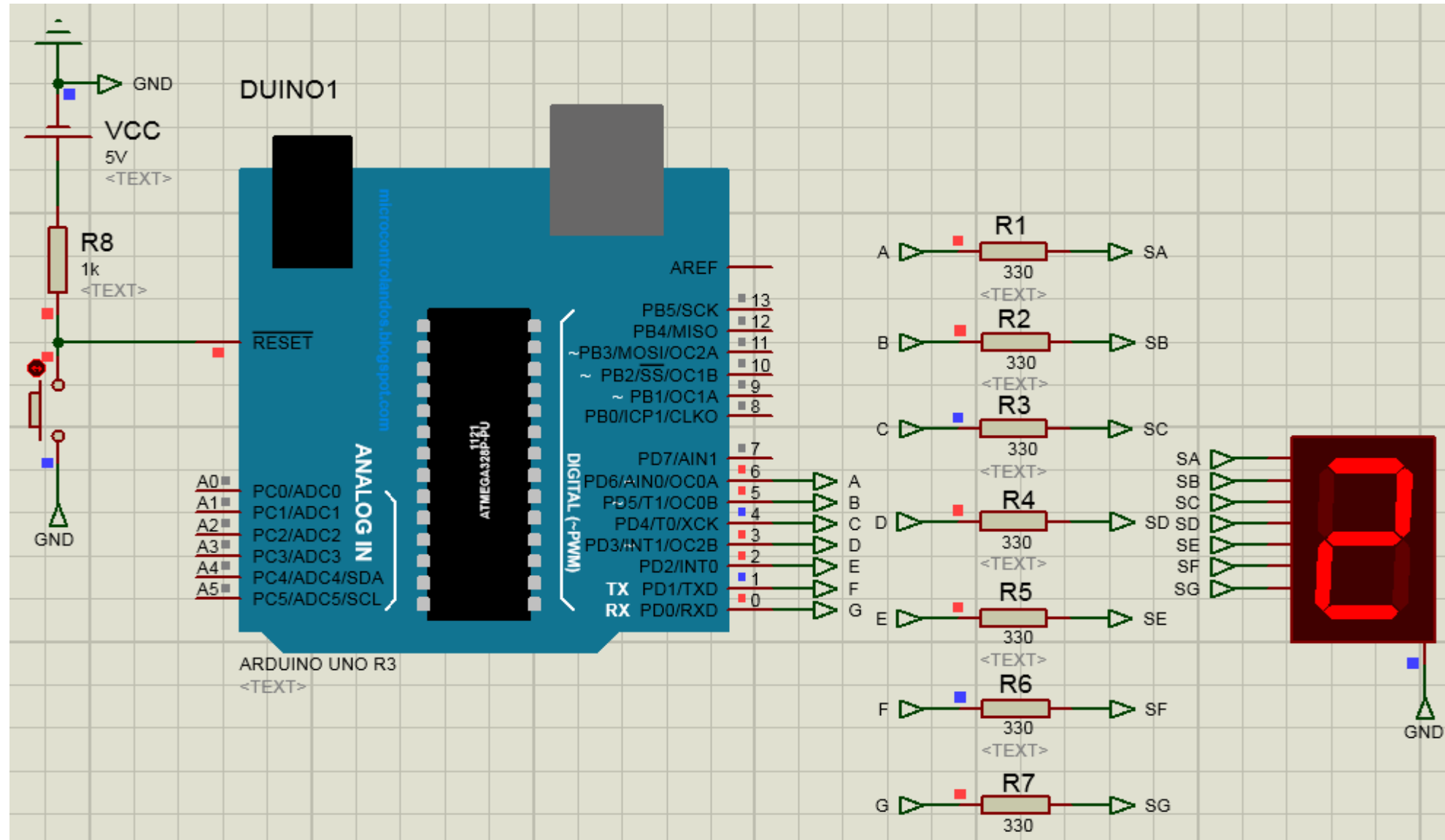
```
  Etat_Led = !Etat_Led;
```

```
}
```

EXERCICE VI: COMPTAGE VIA UN AFFICHEUR_7 SEG

TAF : réalisez et programmez un circuit à base d'ARDUINO permettant de compter de zéro à neuf tant que le circuit est alimenté. L'affichage des digits sera réalisé en utilisant un afficheur 7 segments à cathode commune.

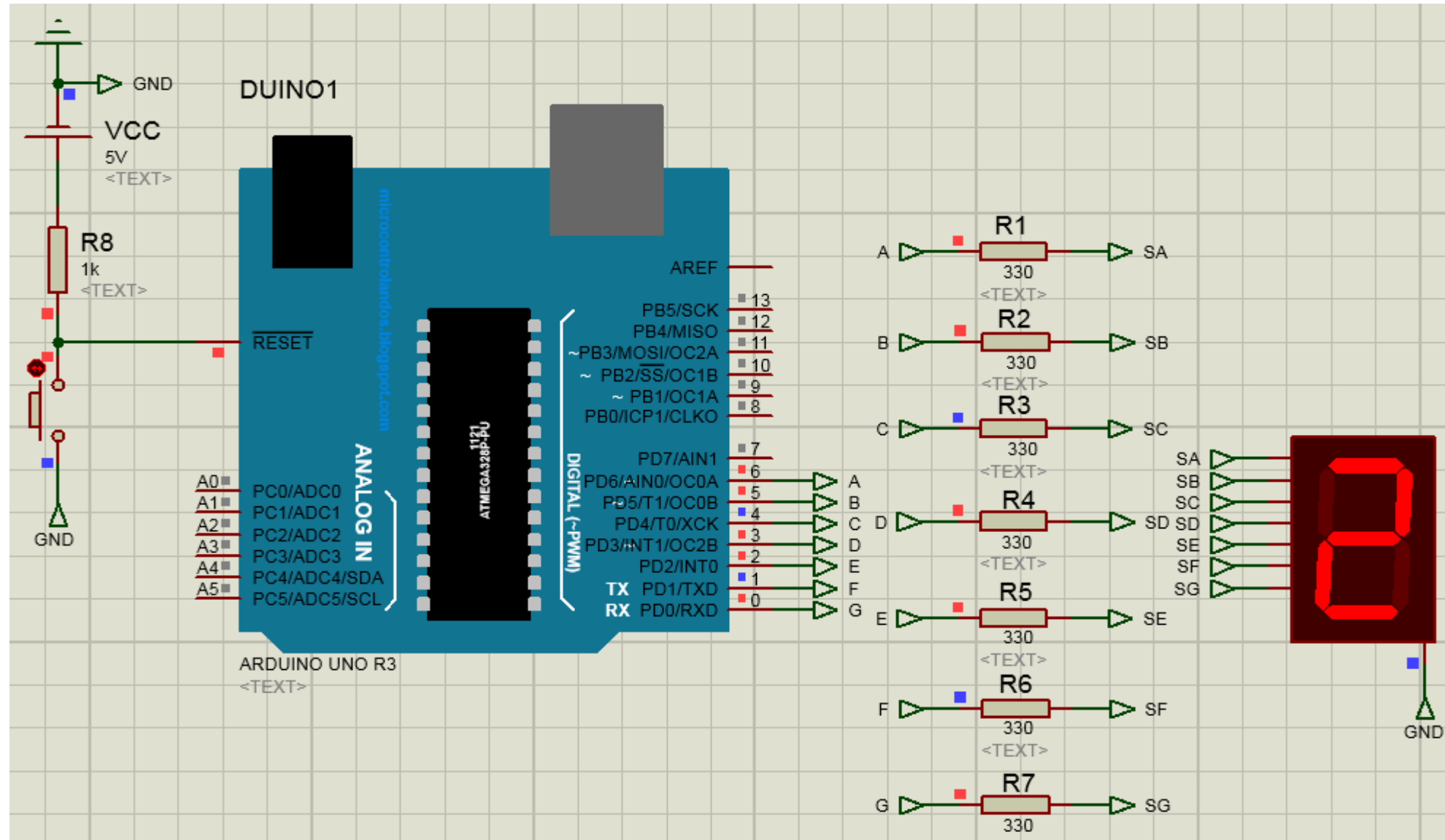
EXERCICE VI_SOLUTION



EXERCICE VII: COMPTAGE/DÉCOMPTAGE 7_SEG

TAF : réalisez et programmez un montage à base d'ARDUINO permettant de compter de zéro à quinze et décompter de quinze à zéro tant que le circuit est alimenté. L'affichage des digits sera réalisé en utilisant un afficheur 7 segments.

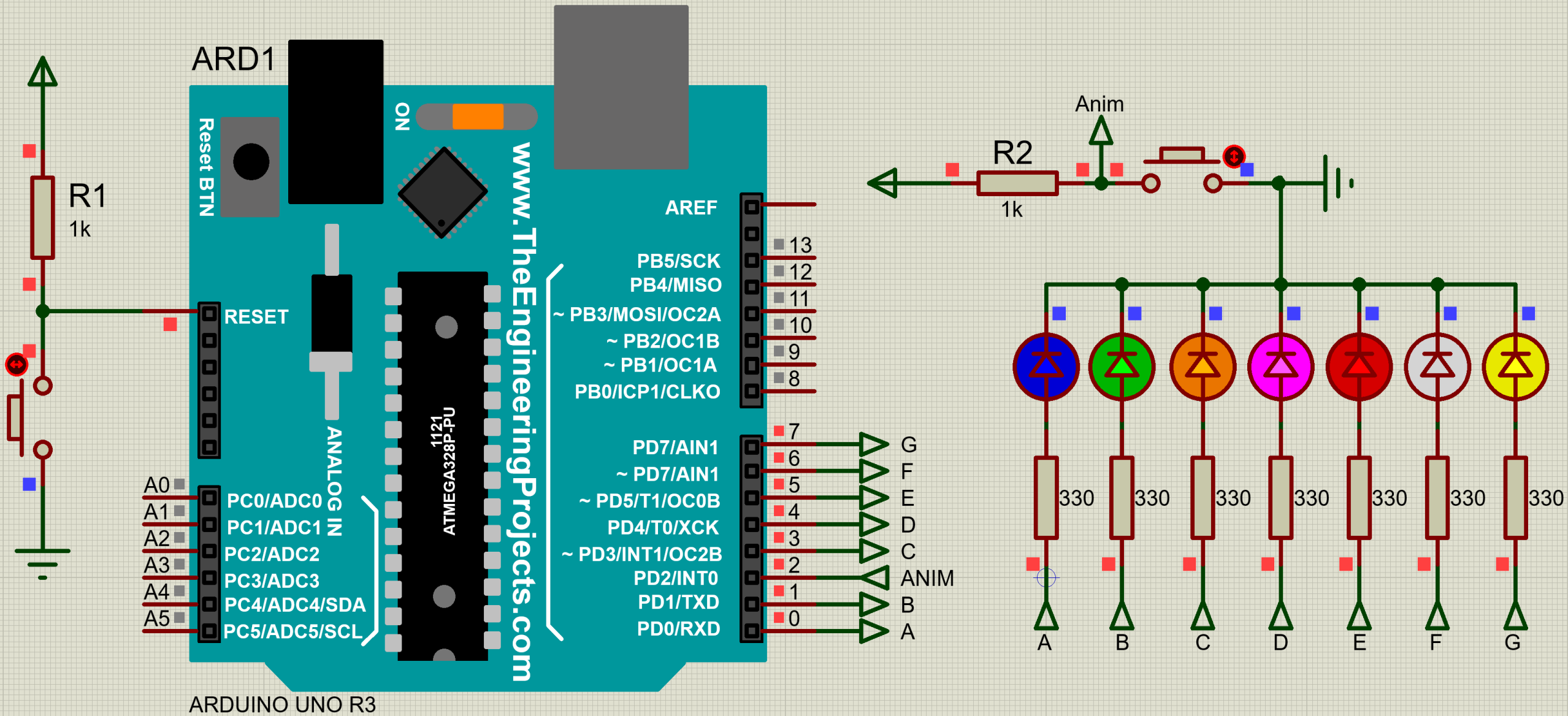
EXERCICE VII_SOLUTION



EXERCICE VIII: CHENILLARD MULTIPLE

TAF : réalisez et programmez un circuit à base d'ARDUINO permettant de réaliser un chenillard multiple sur 7 LEDs. Il s'agit d'un chenillard constitué de 7 différents designs. Initialement, les LEDs sont éteintes. L'appui et relâche d'un bouton poussoir placé sur la broche 2 permet de démarrer le chenillard. Chaque appui et relâche du bouton poussoir permet de passer d'un design à un autre. Au huitième appui, les LEDs s'éteignent.

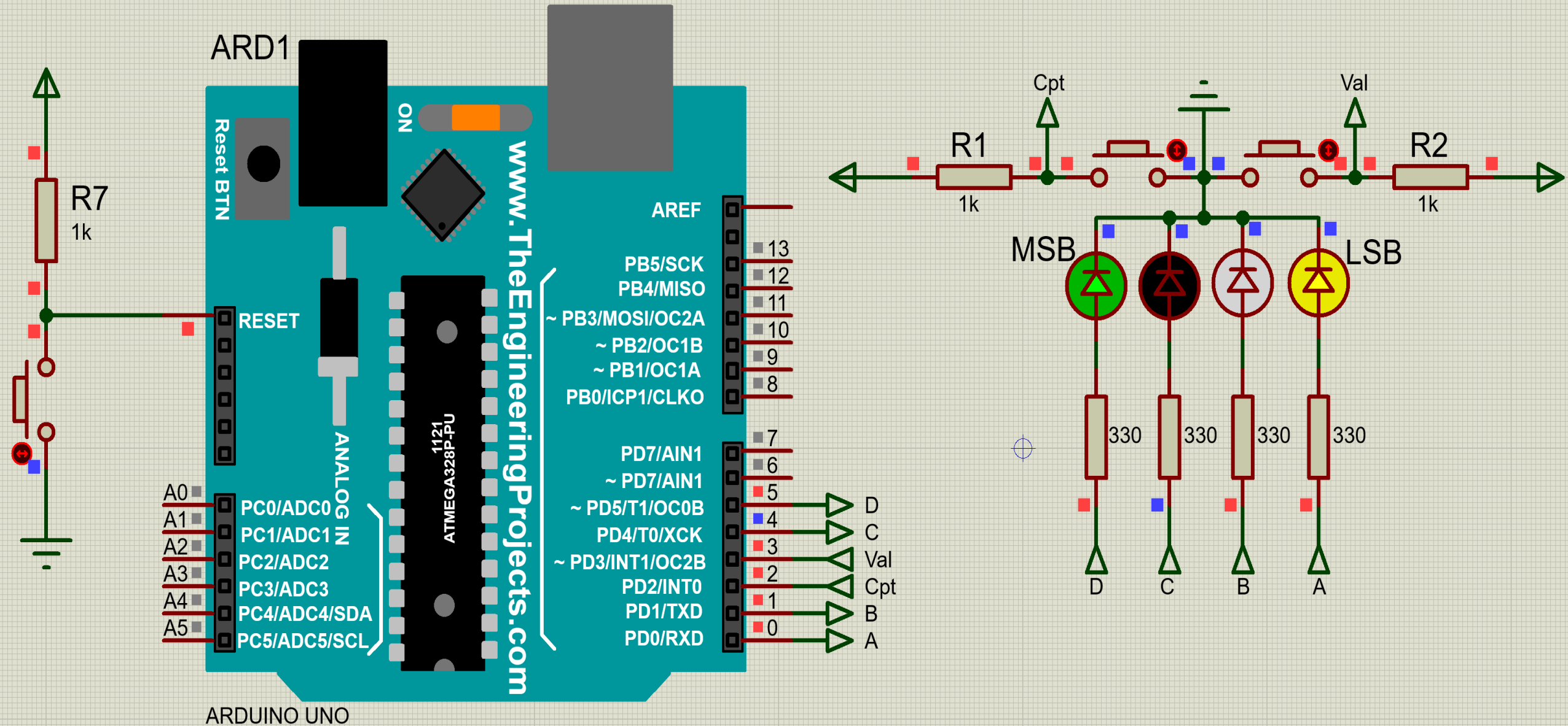
EXERCICE VIII_SOLUTION



EXERCICE IX: TRANSCODEUR

TAF : réalisez et programmez un montage à base d'ARDUINO permettant de transcrire une valeur décimale entrée en base binaire via des LEDs. Le système est constitué de 2 boutons poussoirs dont l'un appelé C_{pt} permettant d'enregistrer le nombre d'appuis et de relâche du bouton poussoir. Le second appelé V_{al} permet de valider le nombre d'appuis capturés par C_{pt} en l'affichant sur 4 LEDs connectées au système. De ce fait, la valeur la plus élevée à afficher est 15 (provoquant l'allumage des 4 LEDs).

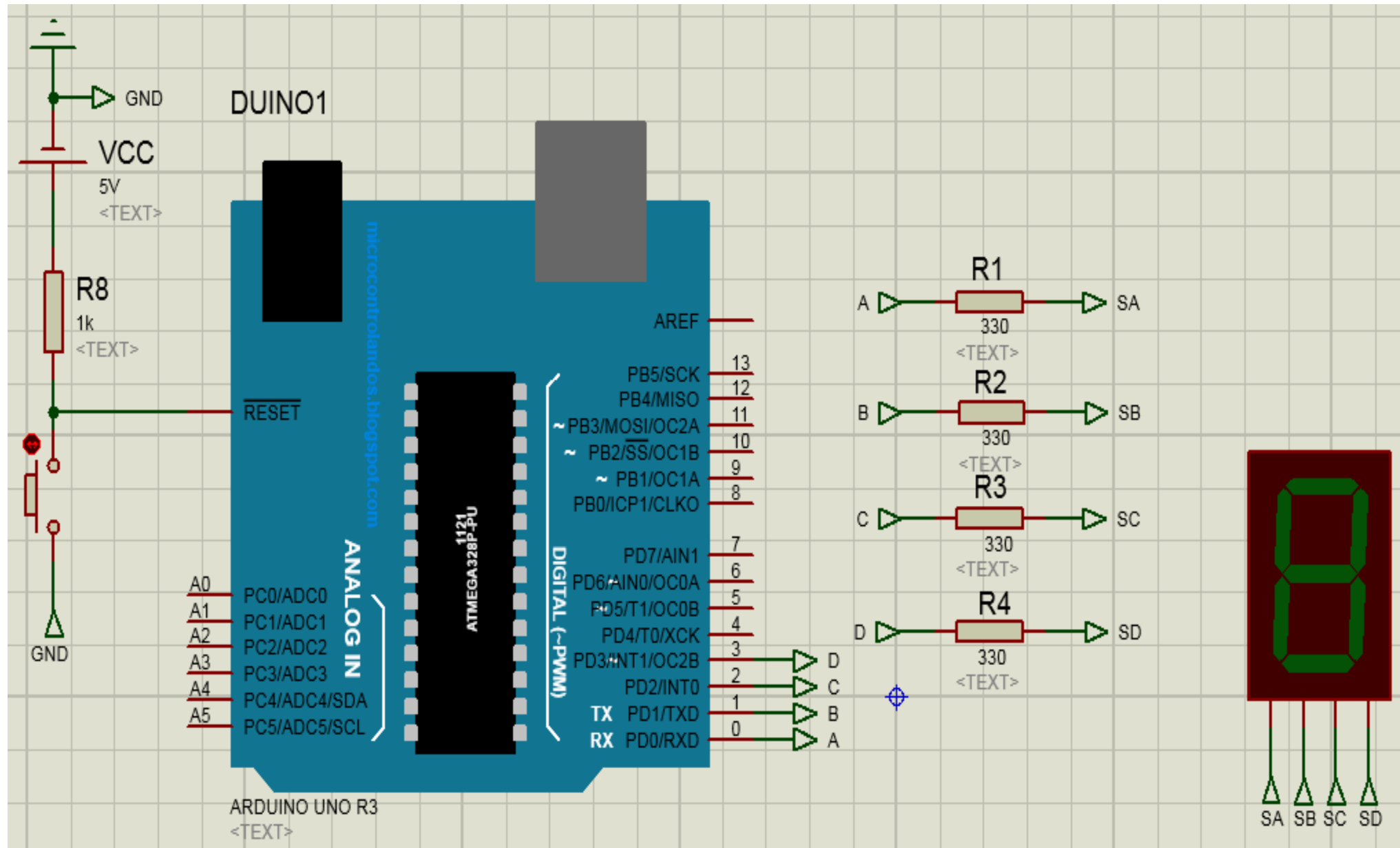
EXERCICE IX_SOLUTION



EXERCICE X: COMPTAGE/DÉCOMPTAGE_BCD

TAF : réalisez et programmez un montage à base d'ARDUINO permettant de compter de zéro à quinze et décompter de quinze à zéro tant que le circuit est alimenté. L'affichage des digits sera réalisé en utilisant un afficheur BCD.

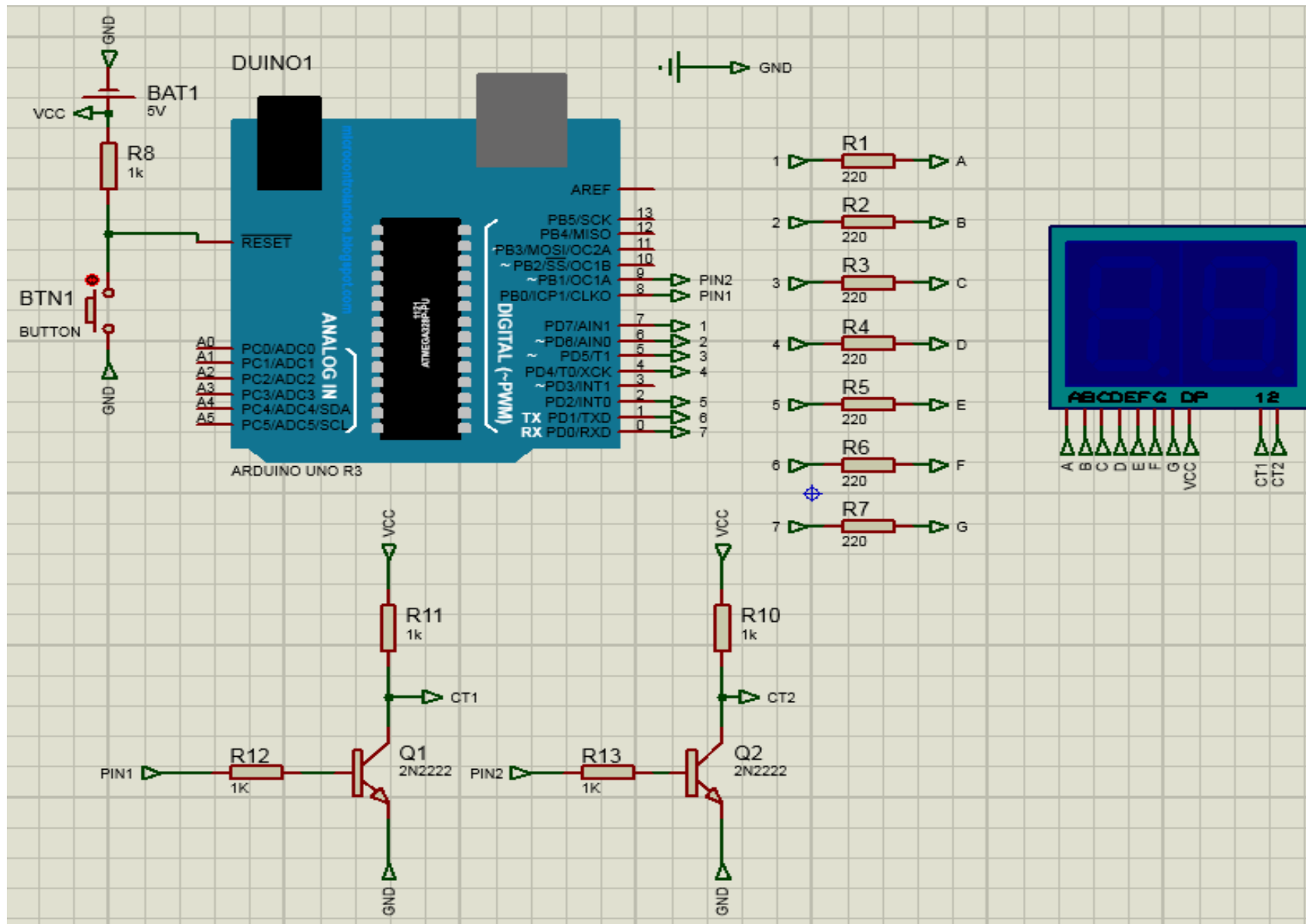
EXERCICE X_SOLUTION



EXERCICE XI: COMPTAGE_2X7SEG

TAF : réalisez et programmez un montage à base d'ARDUINO permettant de compter de zéro à quatre-vingt-dix-neuf continuellement.

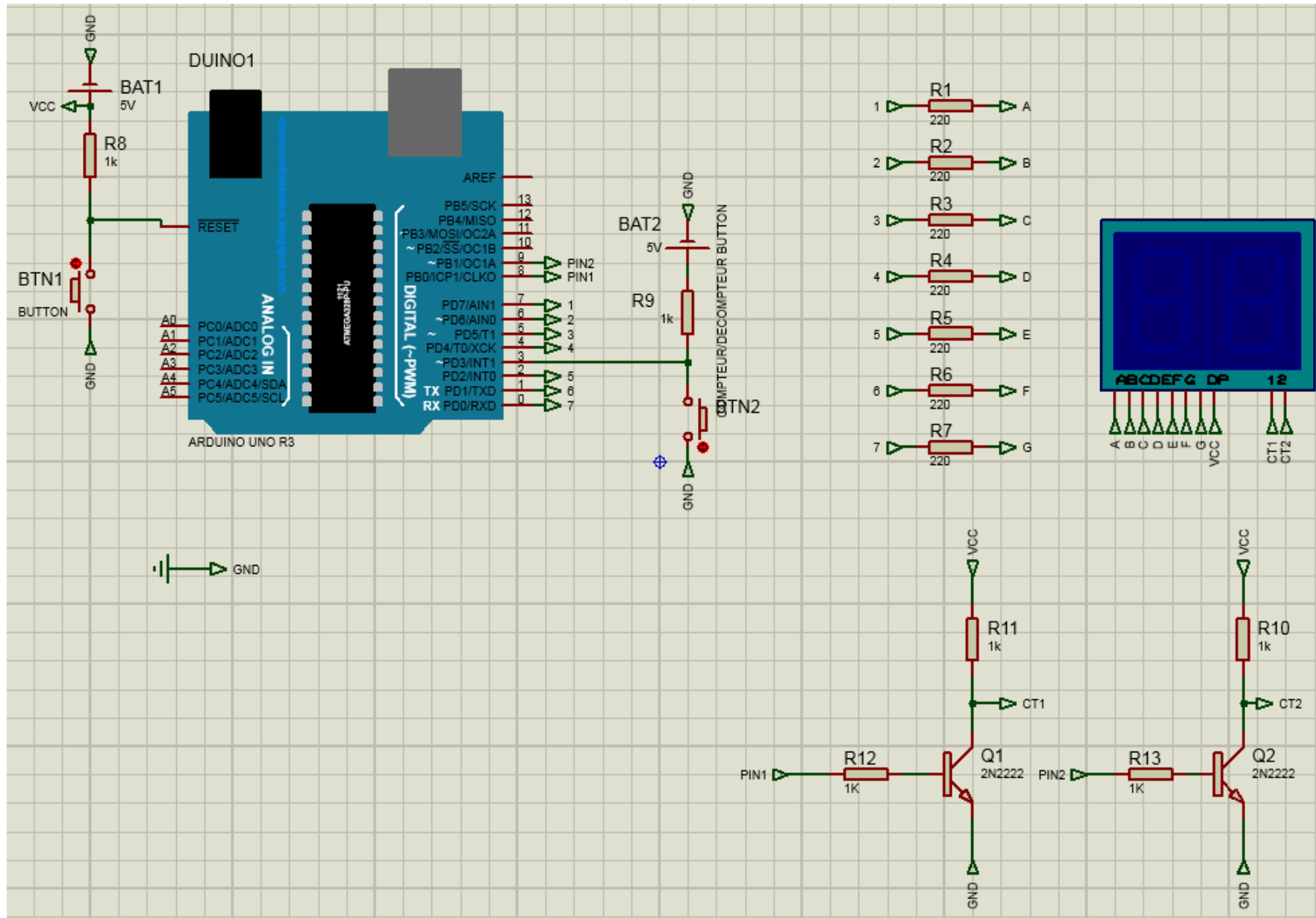
EXERCICE XI_SOLUTION



EXERCICE XII: COMPTAGE/DÉCOMPTAGE_2X7 SEG

TAF : réalisez et programmez un montage à base d'ARDUINO permettant de soit compter de 00 à 99, soit de décompter suivant l'appui d'un bouton poussoir.

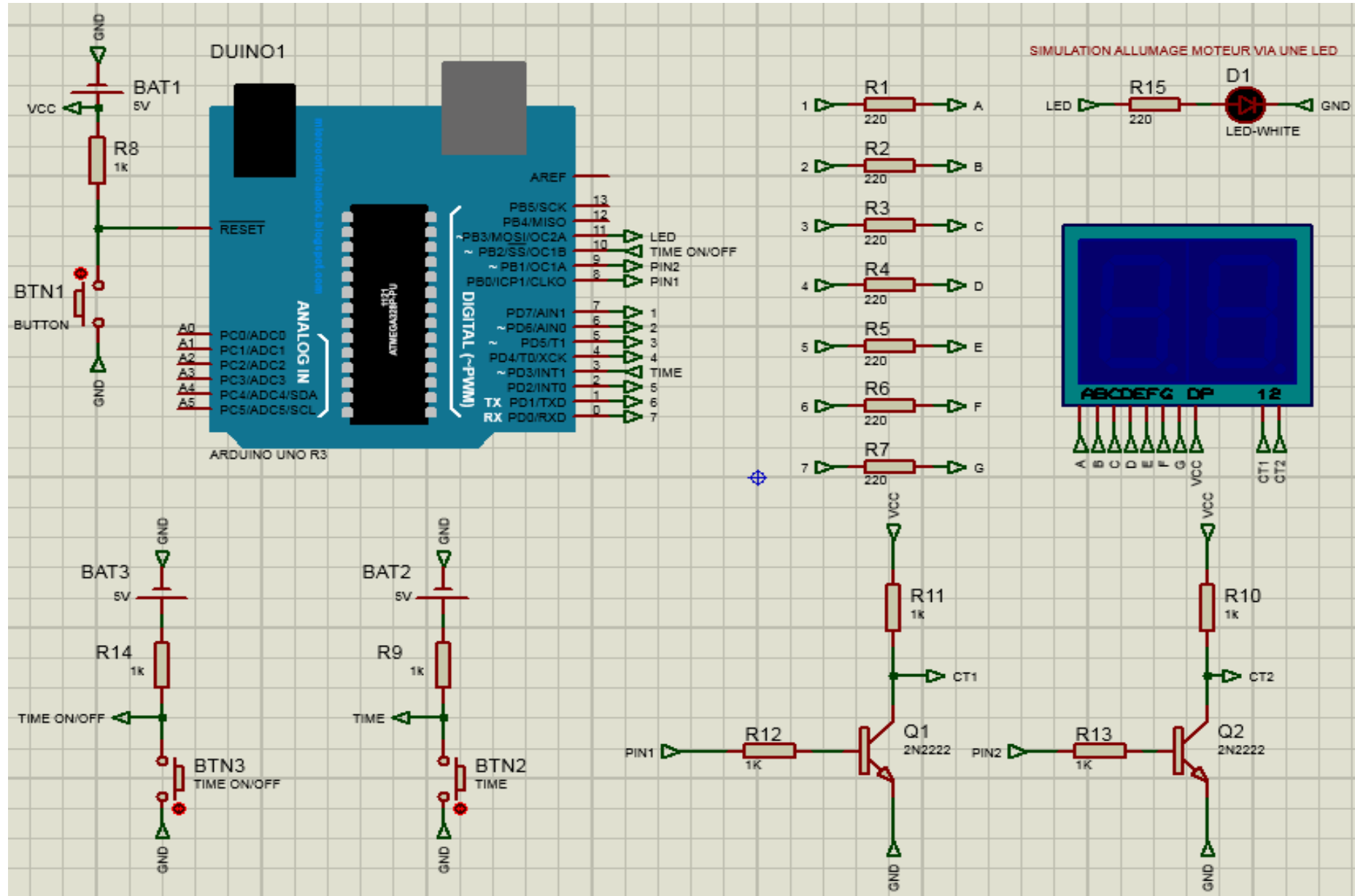
EXERCICE XII_SOLUTION



EXERCICE XIII: CONTRÔLE MOTEUR

TAF : réalisez et programmez un montage à base d'ARDUINO permettant de démarrer un engin après un temps qui pourra être modifié par l'utilisateur. L'engin en fonctionnement sera remplacé par le clignotement d'une LED. Le temps possible devra s'étendre de 00 à 99.

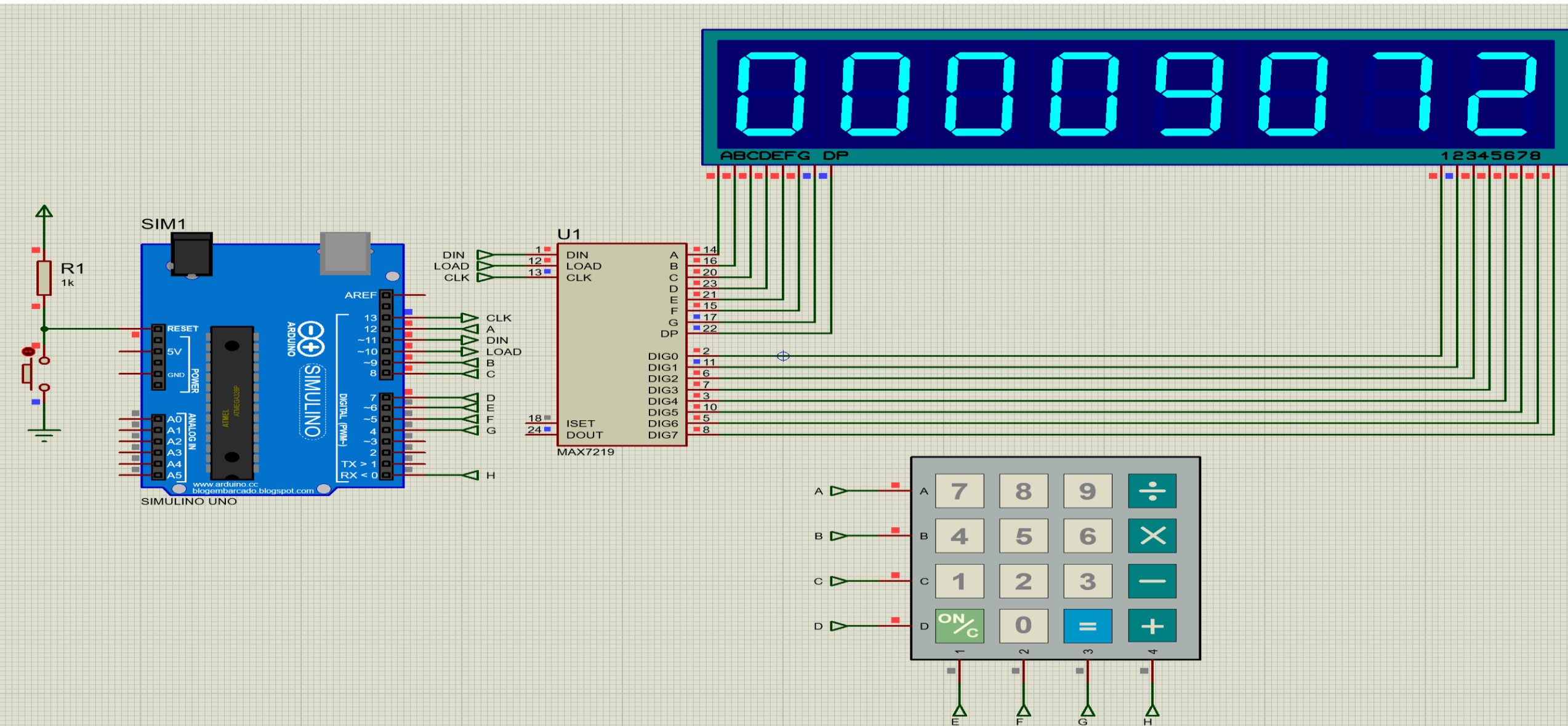
EXERCICE XIII_SOLUTION



EXERCICE XIV: CALCULATRICE

TAF : réalisez et programmez un montage à base d'ARDUINO permettant de réaliser les opérations mathématiques de base. Afin de faciliter l'affichage des données sur un ensemble de huit afficheurs 7 segments, veuillez utiliser le composant MAX7219.

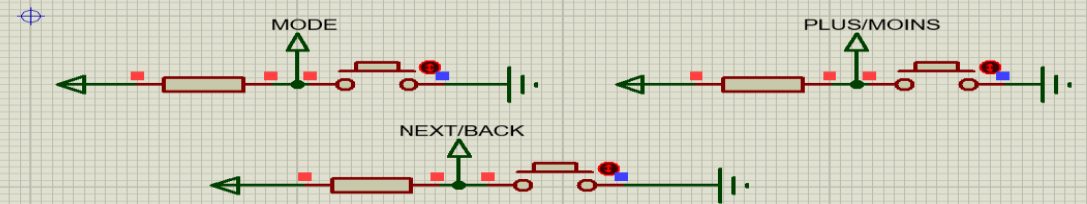
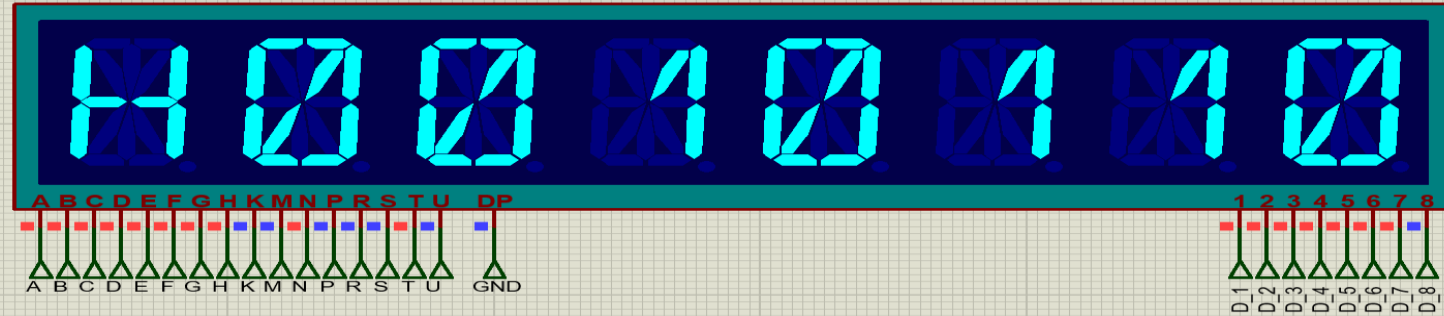
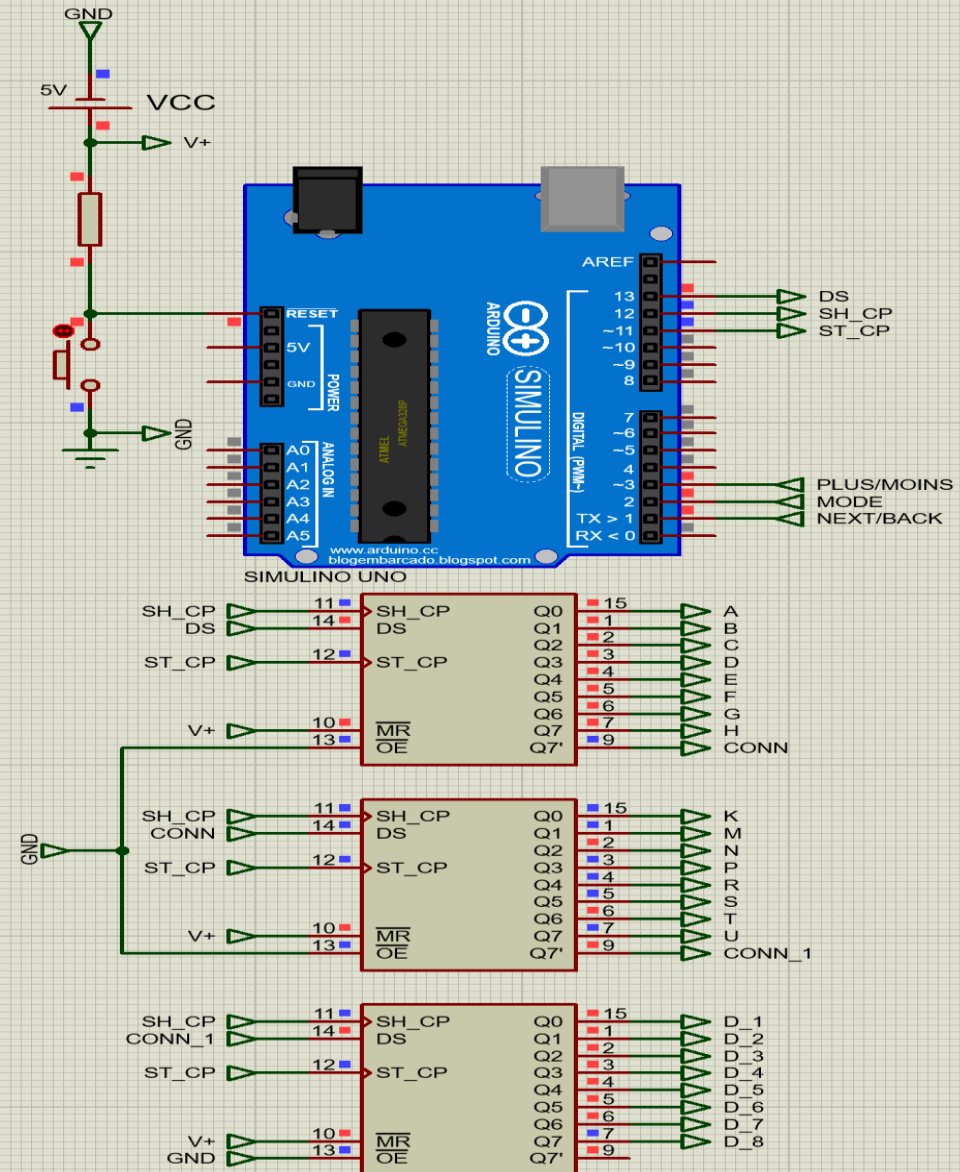
EXERCICE XIV: CALCULATRICE



EXERCICE XV: PRINCIPE DE HAMMING

TAF : réalisez et programmez un montage à base d'ARDUINO permettant de calculer une information à transmettre selon le code de Hamming. Cela devra être affiché sur un ensemble de 8 afficheurs 16 segments. Le circuit devra permettre d'entrer l'information de départ, puis afficher l'information à transmettre. Les bits de contrôle seront automatiquement calculés par le système. Afin de faciliter l'affichage des données, l'on pourra utiliser le composant 74HC595.

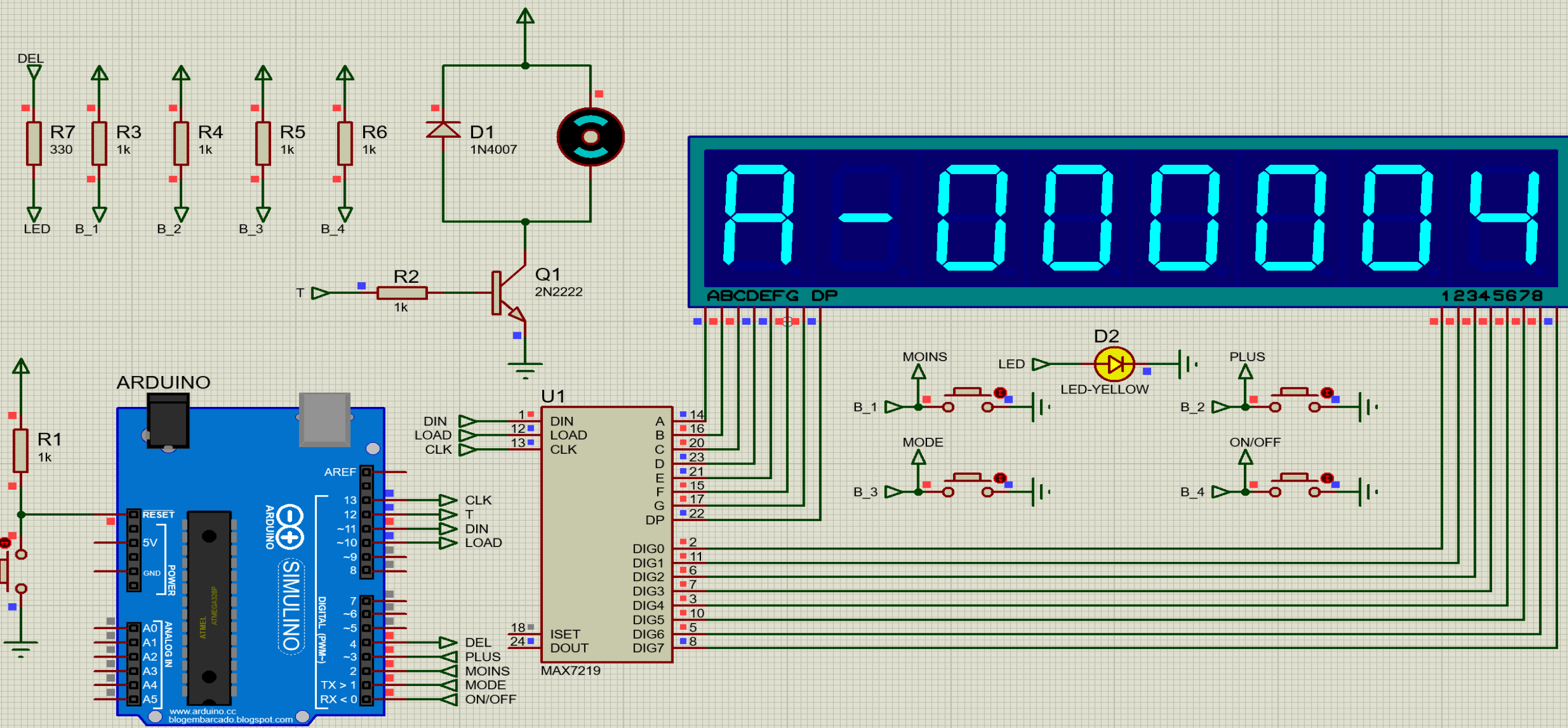
EXERCICE XV_SOLUTION



EXERCICE XVI: MARCHE ET ARRÊT D'UN MOTEUR

TAF : réalisez et programmez un montage à base d'ARDUINO permettant de contrôler le fonctionnement d'un moteur. Le système devra permettre de modifier (par l'utilisateur) le temps de marche et d'arrêt du moteur. Initialement, le système est éteint. Un appui et relâche sur un bouton poussoir devra l'allumer. Après la marche du moteur, le système devra automatiquement s'éteindre. Après appui du bouton poussoir prévu pour démarrer le système, le moteur devra être inactif pendant un temps A, puis se mettre en marche pendant un temps B. Une LED témoin sera placée et allumée pendant le fonctionnement du système. Pendant, le temps d'arrêt du moteur l'on devra voir le décomptage de ce temps. Idem lorsque le moteur est en marche.

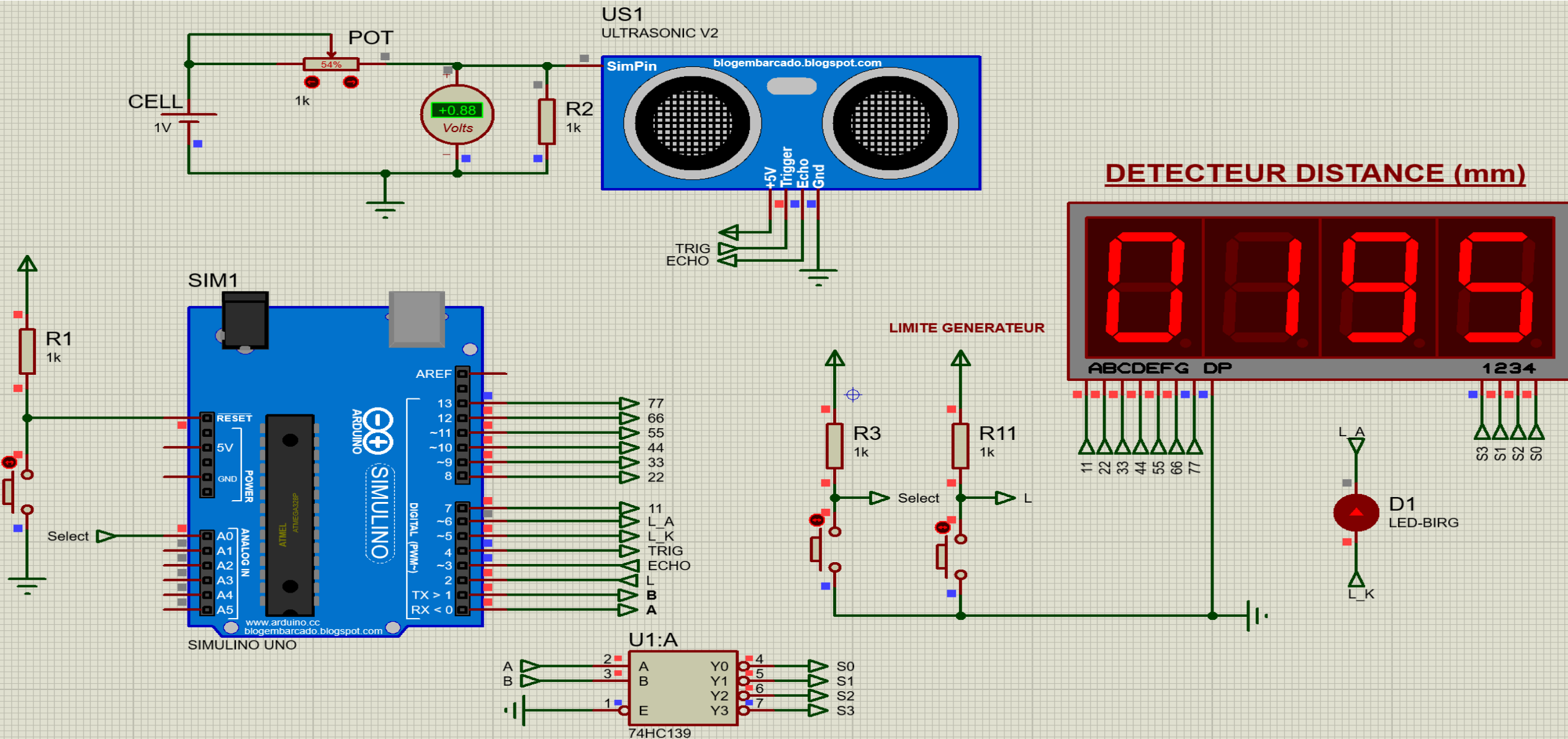
EXERCICE XVI_SOLUTION



EXERCICE XVII: DÉTECTEUR DE DISTANCE

TAF : réalisez et programmez un montage à base d'ARDUINO permettant d'allumer une LED selon la distance séparant un obstacle du système. En pratique, la LED pourrait être remplacée par une porte ou tout autre système de sécurité.

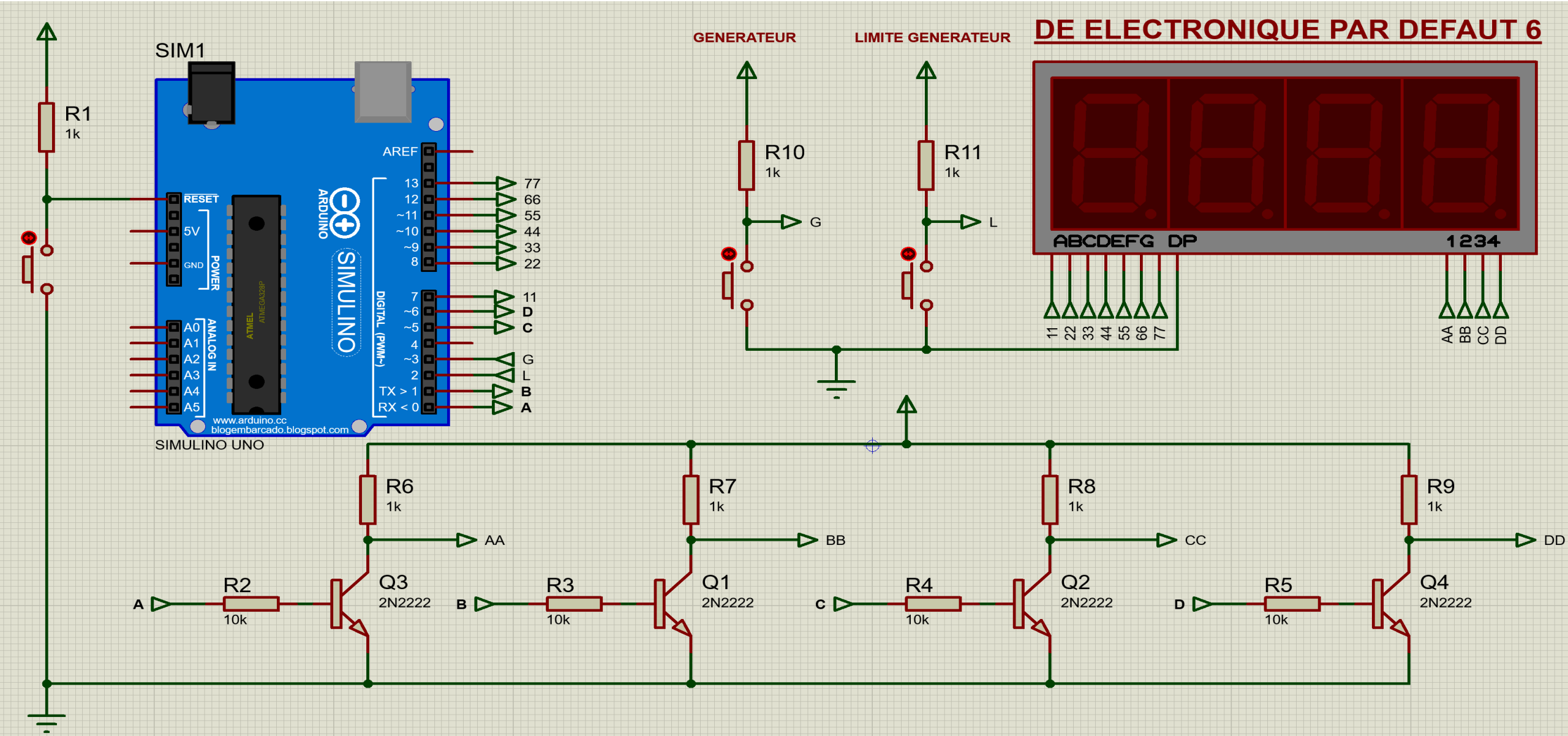
EXERCICE XVII_SOLUTION



EXERCICE XVIII: DE ÉLECTRONIQUE

TAF : réalisez et programmez un montage à base d'ARDUINO simulant le fonctionnement d'un DE (d'où l'appellation DE électronique). Lorsque le système est actionné, un décomptage (modifiable) est lancé. Après quoi, une valeur aléatoire est affichée sur un ensemble de 4 afficheurs 7 segments. La limite d'appartenance de la valeur aléatoire est aussi modifiable.

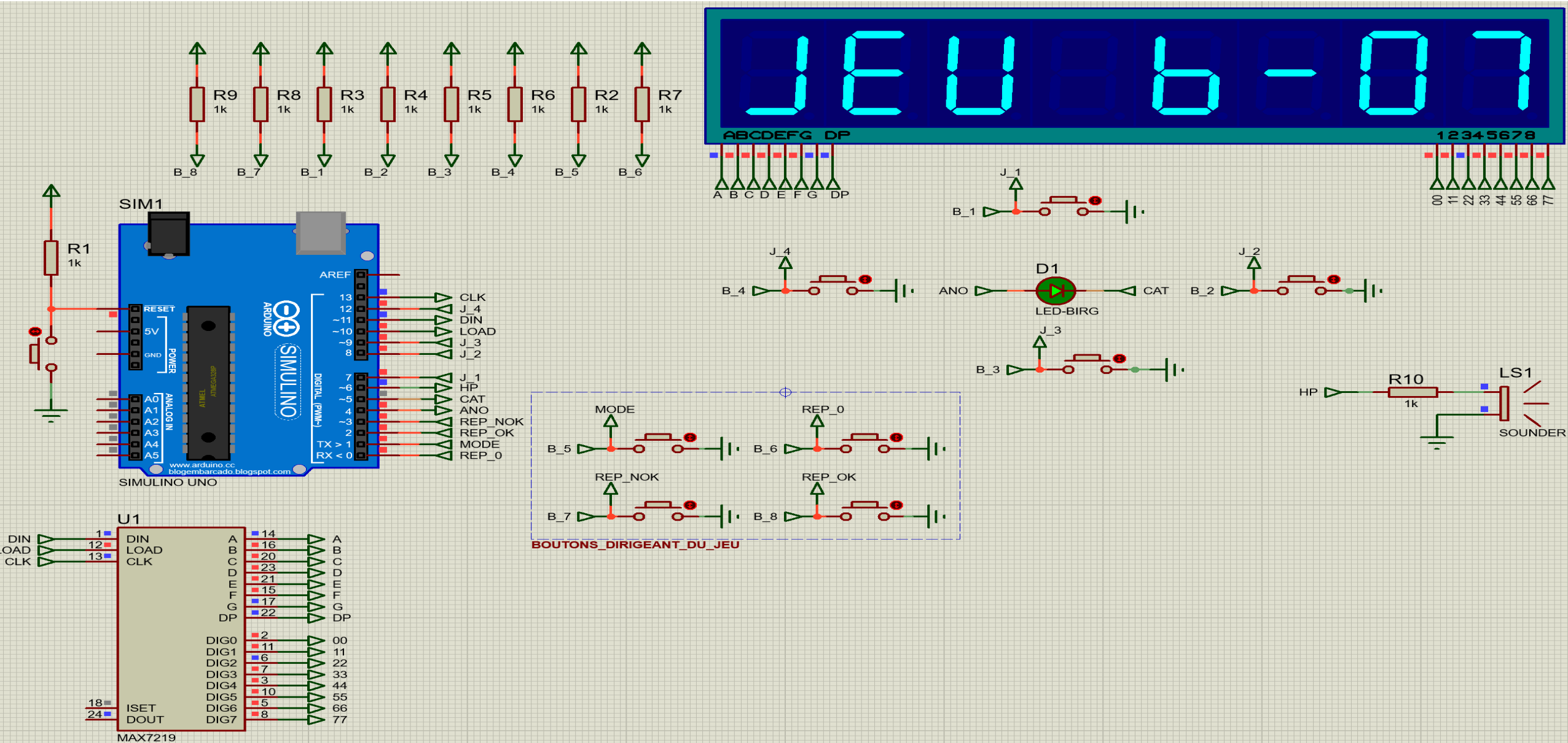
EXERCICE XVIII_SOLUTION



EXERCICE XIX: JEU QUESTIONS-RÉPONSES

TAF : réalisez et programmez un montage à base d'ARDUINO simulant le fonctionnement du jeu appelé QUESTIONS POUR UN CHAMPION. Le système accepte 4 joueurs. Lorsque la main est passée à un joueur à la suite de l'appui du bouton poussoir mis à sa disposition (un bip sonore est retenti), un décomptage du temps (modifiable par le dirigeant du jeu) qui lui est alloué est réalisé (la lettre attribuée au joueur et le temps de décomptage sont affichés). Après avoir répondu à la question du dirigeant du jeu, ce dernier pourra valider ou invalider la réponse du joueur (une réponse jugée correcte se verra affecter une note aussi modifiable par le dirigeant du jeu; tout comme une réponse jugée incorrecte. Une LED témoin est utilisée pour informer le joueur sur sa réponse donnée. Lorsque la LED est verte, cela sous-entend que la réponse donnée est correcte. Lorsque cette dernière est rouge, cela sous-entend que la réponse donnée est incorrecte. L'état de la LED est fonction de l'action du dirigeant sur les boutons poussoirs prévus pour la validation ou l'invalidation d'une réponse donnée. La note finale d'un joueur est donnée par la différence entre les notes des réponses correctes et celles incorrectes). À la fin du jeu, le système affichera les résultats du jeu (le rang et le nombre total de points de chaque joueur). Tout ceci sera affiché sur un ensemble de 8 afficheurs 7 segments. L'affichage des données sur l'ensemble des afficheurs pourra être allégé en utilisant le composant MAX 7219.

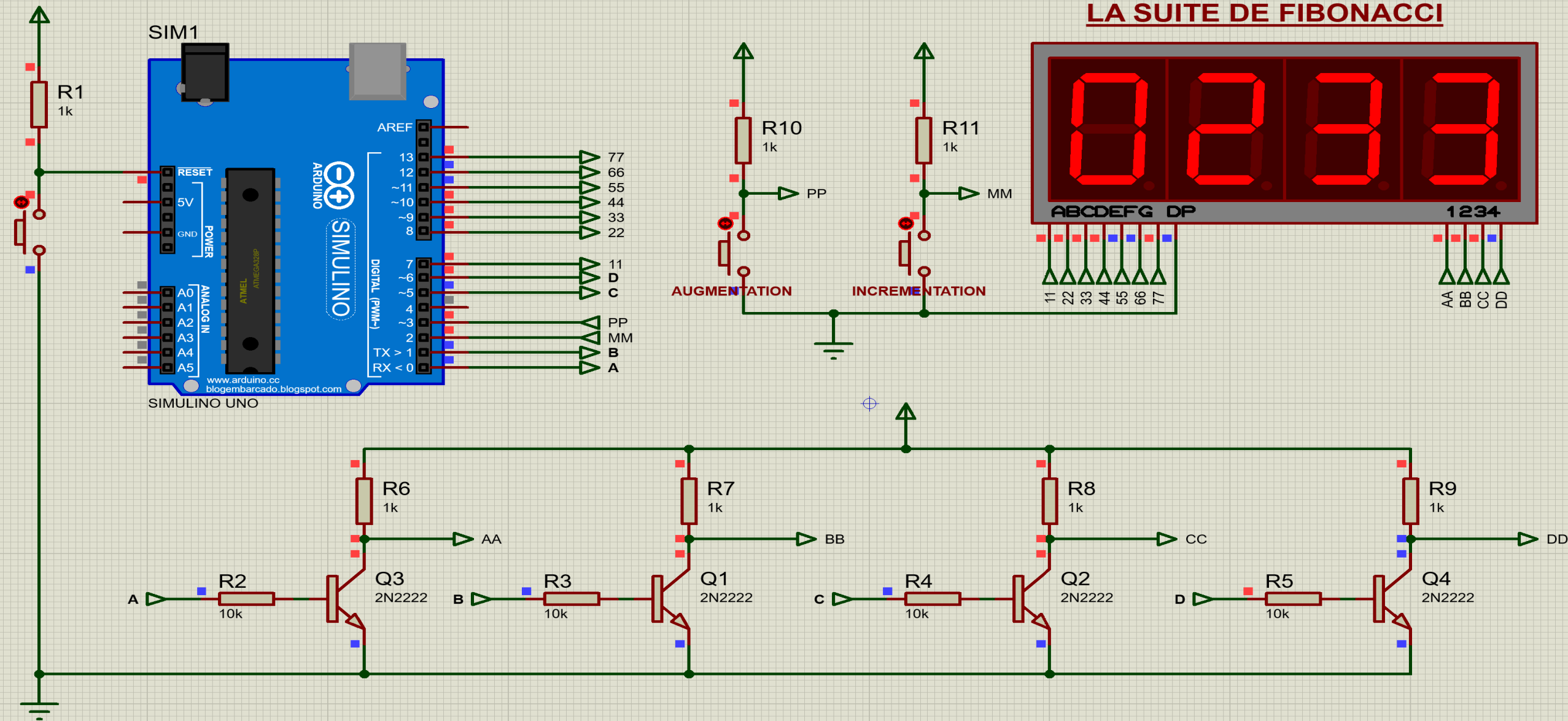
EXERCICE XIX_Solution



EXERCICE XX: LA SUITE DE FIBONACCI

TAF : réalisez et programmez un montage à base d'ARDUINO permettant de calculer et d'afficher les valeurs correspondant à la suite de Fibonacci. Un appui et relâche sur un bouton poussoir permet d'afficher la valeur suivante tandis qu'un autre permettant d'afficher la valeur précédente. Les données devront être affichées sur un ensemble de 4 afficheurs 7 segments.

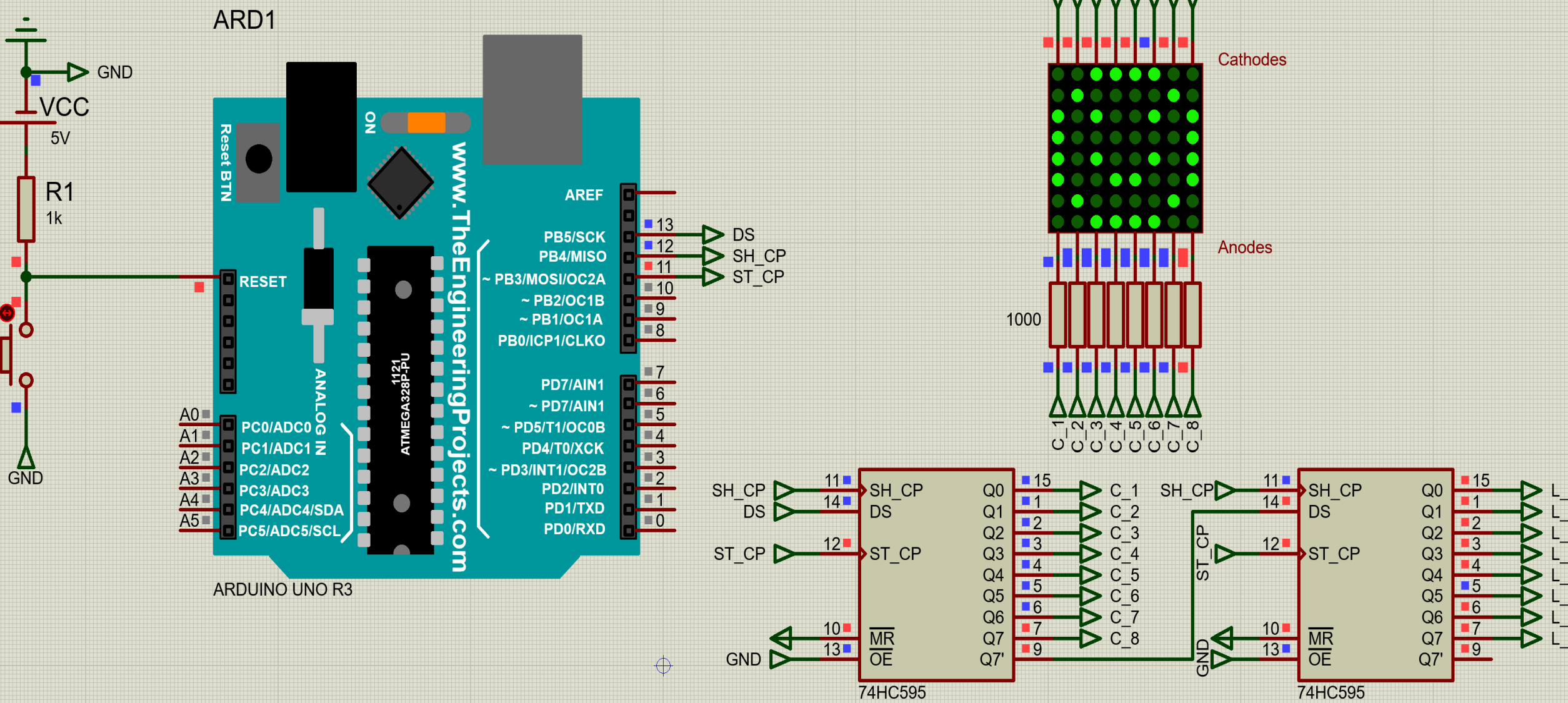
EXERCICE XX_SOLUTION



EXERCICE XXI: PICTOGRAMME

TAF : réalisez et programmez un montage à base d'ARDUINO permettant d'afficher des pictogrammes sur une matrice de LEDs de taille 8x8. Le passage d'un pictogramme à un autre est réalisé après que celui-ci a clignoté 3 fois (le quatrième clignotement doit correspondre à celui du pictogramme suivant) après avoir été affiché (sans clignoter) pendant 5 secondes. Afin de limiter le nombre de broches (à utiliser) de la carte Arduino, veuillez utiliser le composant 74HC595.

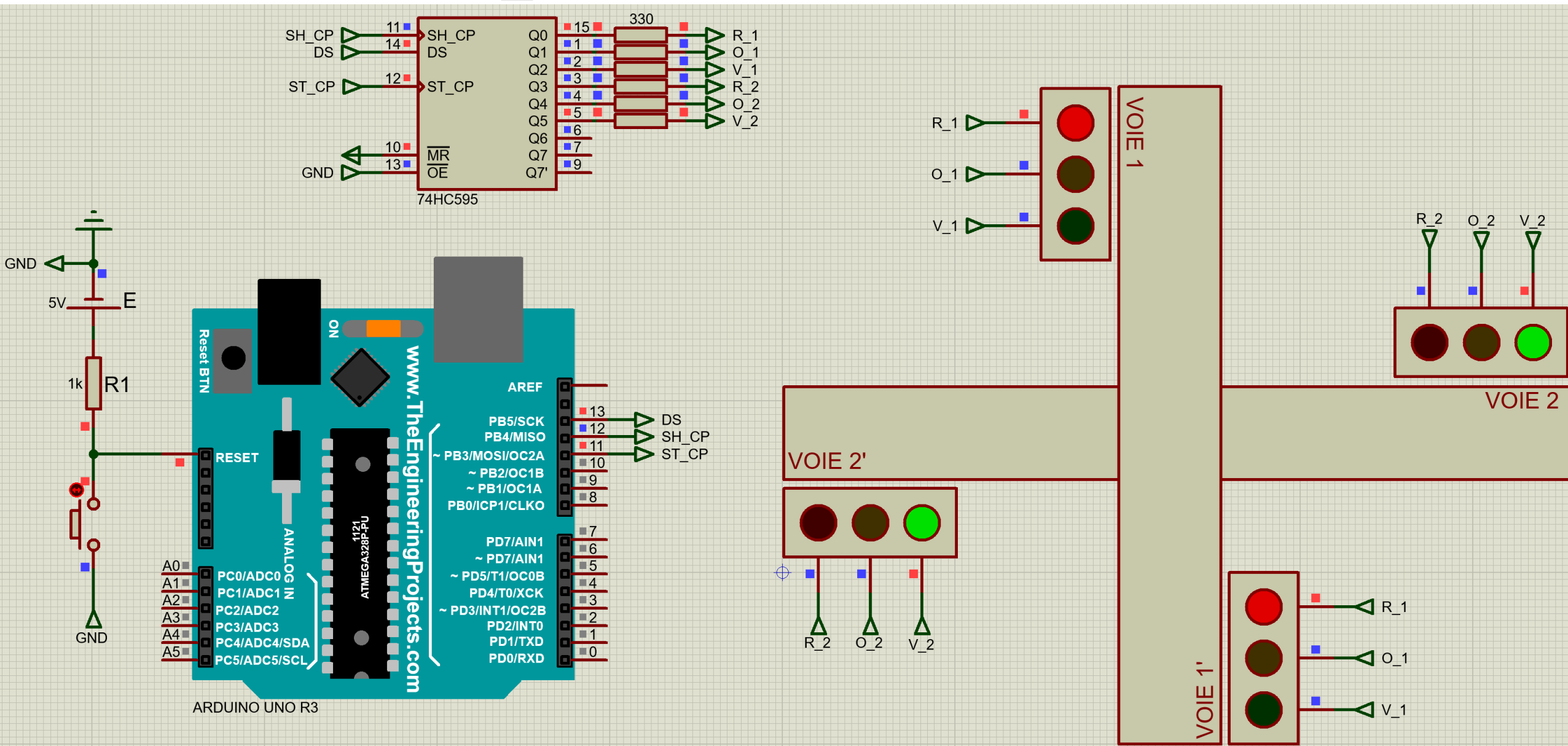
EXERCICE XXI_SOLUTION



EXERCICE XXII: FEUX TRICOLORES_VERSION 1

TAF : réalisez et programmez un montage à base d'ARDUINO décrivant le fonctionnement d'un feu tricolore à un carrefour. Voir diapo suivante pour une mise en route.

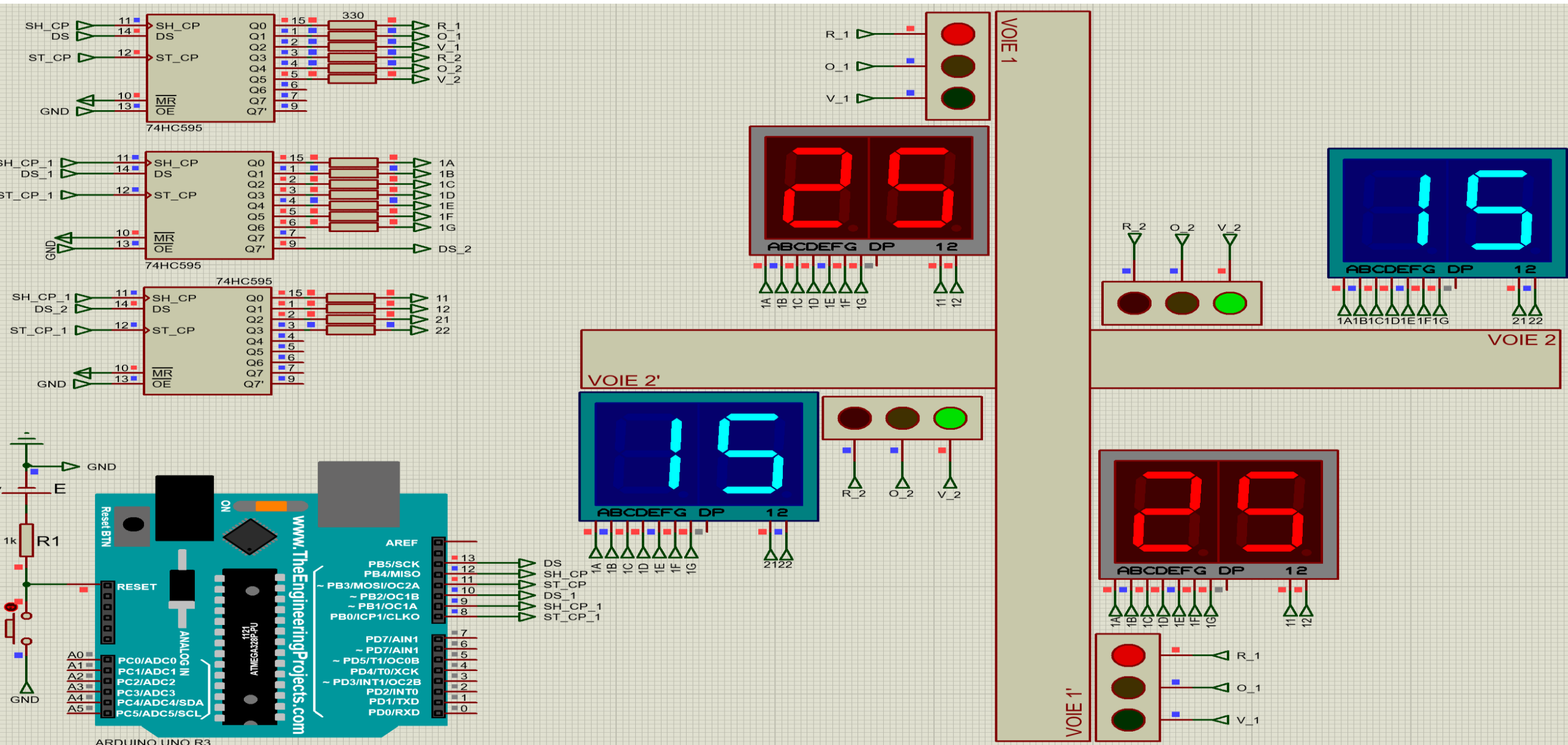
EXERCICE XXII_SOLUTION



EXERCICE XXIII: FEUX TRICOLORES_VERSION 2

TAF : réalisez et programmez un montage à base d'ARDUINO décrivant le fonctionnement d'un feu tricolore à un carrefour. Cette version est différente de la précédente en ce qui concerne l'affichage de la durée de chaque feu. Voir diapo suivante pour une mise en route.

EXERCICE XXIII_SOLUTION



FIN DU COURS

MERCI POUR VOTRE ATTENTION

